

MotorX

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing

SOFTWARE ARCHITECTURE DOCUMENT

Version 1.0

PID 11 — Group 23

230616B Silva T.D.R.

230621K Somarathna M.D.A.M.

230633A Thadshakan J.

Mentor: Bhanuka Siriwardana

Department of Computer Science and Engineering
University of Moratuwa

Revision History

Date	Version	Description	Author
05/Aug/2026	0.1	Initial Software Architecture Document structure derived from SRS v1.0 and Project Proposal	Group 23
07/Aug/2026	0.8	Completed major architectural views and system diagrams	Group 23
09/Aug/2026	1.0	Finalized Software Architecture Document	Group 23

Contribution

Member Name	Index Number	Main Responsibility	Sections / Work
T.D.R. Silva	230616B	Architecture & System Design	Use-Case View; Logical View; part of Process View; Implementation View; Data View; UML usecase diagrams, activity diagrams and architecture diagrams
M.D.A.M. Somarathna	230621K	Process & System Design	ETL and inventory architecture; part of Process View; Deployment View; sequence diagrams; background processing
J. Thadshakan	230633A	Architecture & Quality Design	Architectural goals and constraints; part of Implementation/Data View; quality and performance; supporting diagrams and review

Table of Contents

1. Introduction.....	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Definitions, Acronyms, and Abbreviations.....	1
1.4 References.....	2
1.5 Overview.....	2
2. Architectural Representation.....	3
2.2 Overall System Architecture.....	3
3. Architectural Goals and Constraints	5
Architectural Goals	5
Architectural Constraints	5
4. Use-Case View.....	6
4.1 Actors.....	6
4.2 Actor Use Cases	6
4.2.1 Buyer Use Cases	6
4.2.2 Dealer Use Cases	7
4.2.3 Administrator Use Cases.....	9
4.3 Use-Case Realizations	11
4.3.1 User Authentication	11
4.3.2 Dealer Uploads Vehicle Inventory.....	11
4.3.3 Buyer Searches Using Natural Language	12
4.3.4 Buyer Searches Using Structured Filters	13
4.3.5 Administrator Approves Dealer Registration.....	13
4.3.6 Dealer Manages Vehicle Listing and Images.....	14
4.3.7 Administrator Monitors the System.....	15
5. Logical View.....	15
5.1 Overview.....	15
5.2 Architecturally Significant Design Packages.....	16
5.3 Core Domain Model	20
5.3.1 Identity & User Domain.....	20
5.3.2 Dealer / Marketplace Domain	21
5.3.3 Inventory / Search Domain	21
5.4 Important Domain Relationships	21
6. Process View.....	22
6.1 Process Organisation and Communication	22
6.2 Dealer Bulk Inventory Upload Activity.....	22
6.3 ETL Pipeline Processing Activity.....	24

6.4 Dealer Uploads Inventory Sequence.....	25
6.5 User Login Sequence	25
6.6 Buyer Natural-Language Search Sequence.....	26
6.7 Buyer Structured Filter Search Sequence	27
6.8 Semantic Embedding Generation Sequence	28
6.9 Administrator Approves Dealer Registration Sequence	29
6.10 Notification Delivery Sequence	29
6.11 Upload Job State	30
6.12 Listing / Rejected Record State.....	31
7. Deployment View	31
Node Descriptions.....	32
8. Implementation View.....	33
8.1 Overview.....	33
8.2 Layers.....	35
8.2.1 Presentation Layer.....	35
8.2.2 API / Controller Layer	36
8.2.3 Business / Service Layer.....	38
8.2.4 ETL / Processing Layer.....	39
8.2.5 Data Access Layer	41
8.3 Package Structure.....	42
9. Data View (Optional).....	44
9.1 Entity-Relationship Model.....	45
9.2 Data-Store Responsibilities.....	46
10. Size and Performance	47
10.1 API Responsiveness.....	48
10.2 Search Performance	48
10.3 Bulk Inventory Processing.....	49
10.4 Background Job Performance	49
10.5 Database Size and Retrieval.....	50
10.6 Object Storage.....	51
10.7 Scalability	51
10.8 Availability	51
11. Quality.....	52
12. References.....	53

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

1. Introduction

This Software Architecture Document (SAD) provides a comprehensive architectural overview of MotorX, the Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing. It captures and conveys the significant architectural decisions made for the system using multiple architectural views: the Use-Case View, Logical View, Process View, Deployment View, Implementation View and Data View. This document is derived from, and should be read together with, the MotorX Software Requirements Specification (SRS) v1.0 and the Project Proposal (PID 11).

1.1 Purpose

The purpose of this Software Architecture Document is to provide an architectural overview of MotorX and to record the significant design decisions made for the system.

It explains how the buyer/dealer web client, the core backend API, the asynchronous ETL worker, the hybrid search mechanism, the databases, object storage, and external providers (Firebase Authentication, the embedding model service) work together. It also defines the responsibilities and boundaries of the main modules, including identity and access management, dealer management, marketplace/listing management, inventory ETL processing, intelligent search, notifications, and administration.

This document will support:

- Development and integration of system components.
- Testing of major workflows and service interactions.
- Deployment and maintenance of the platform, locally and on AWS.
- Evaluation of the selected architecture.
- Future extension or independent scaling of individual components.

The intended audience includes the project development team, the project mentor/supervisor, evaluators, and future maintainers. The SRS defines what the system must do; this document describes how the system is structured to satisfy those requirements.

1.2 Scope

This document applies to the full MotorX system as defined in the SRS: the dealer and inventory-management subsystem, the buyer marketplace and search subsystem, the asynchronous ETL pipeline, and the administration subsystem. It covers the web-based frontend, the modular monolithic backend API, the dedicated ETL worker, the persistent database, object storage, the background job queue, and the hybrid (structured + fuzzy + semantic) search mechanism, across both the local Docker development environment and the AWS production environment.

1.3 Definitions, Acronyms, and Abbreviations

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Definitions, acronyms and abbreviations used throughout this document follow those defined in Section 1.3 of the MotorX SRS v1.0 (e.g., ETL, RBAC, JWT, REST, S3, Semantic Embedding, Hybrid Search). Key terms used specifically in this architectural document are summarized below.

Term	Definition
Modular Monolith	A single deployable backend application whose internal code is organized into clearly separated logical modules (Auth, Dealer, Marketplace, Inventory/ETL, Search, Notifications, Administration).
ETL Worker	A dedicated, independently scalable background process that consumes queued inventory-processing jobs and performs extraction, normalization, validation, deduplication, categorization, enrichment and embedding generation.
Hybrid Search	A search strategy that combines deterministic structured filtering, fuzzy text matching, and semantic vector similarity to rank vehicle listings.
Vector Embedding	A numerical representation of listing or query text used to compute semantic similarity for intelligent search.
Upload Job	A tracked record representing the asynchronous processing lifecycle of one dealer inventory file, from Pending through Processing to Completed/Failed.

1.4 References

- [1] Group 23, "Project Proposal — Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing," PID 11, University of Moratuwa, 05/Jul/2026.
- [2] Group 23, "MotorX — Software Requirements Specification," Version 1.0, University of Moratuwa, 02/Aug/2026.

Diagramming tool: draw.io / Lucidchart /PlantUML

1.5 Overview

The remainder of this document is organized as follows:

Section 2 – Architectural Representation: Introduces the selected architecture and the architectural views used to describe it.

Section 3 – Architectural Goals and Constraints: Describes the security, privacy, performance, reliability, maintainability, deployment, and technology constraints that influence the architecture.

Section 4 – Use-Case View: Presents the major functions available to dealers, buyers, and administrators.

Section 5 – Logical View: Describes the service modules, packages, domain classes, responsibilities, and dependencies.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Section 6 – Process View: Describes runtime workflows using activity, sequence and state diagrams.

Section 7 – Deployment View: Shows the physical and cloud nodes on which the applications and services are deployed.

Section 8 – Implementation View: Describes the implementation components, packages, layers, and interfaces.

Section 9 – Data View: Presents the database structure and the division of responsibilities between MongoDB and Redis.

Section 10 – Size and Performance: Describes the significant capacity and performance considerations.

Section 11 – Quality: Explains how the architecture supports security, privacy, reliability, maintainability, scalability, availability, portability, and auditability.

Section 12 – References: Provides the complete references used in preparing the document.

2. Architectural Representation

Architectural View	Representation
Use-Case View	Describes the major system functions available to dealers, buyers, and administrators using the Dealer, Buyer, and Administrator Use-Case Diagrams.
Logical View	Describes the decomposition of the backend into service modules, packages, and domain classes using the Logical Service Decomposition Diagram and the Core Domain Class Diagram.
Process View	Describes the runtime workflows and interactions among users, interfaces, services, workers, and data stores using activity, sequence, and state diagrams.
Deployment View	Describes the physical and cloud environments on which the applications, services, workers, and data stores are deployed using the Deployment Diagram.
Implementation View	Describes the major software components, layers and their interfaces using the Overall Component Diagram and the Backend Package Diagram.
Data View	Describes the persistent data model and the division of storage responsibilities using the Entity-Relationship Diagram and the MongoDB/Redis Responsibility Diagram.

2.2 Overall System Architecture

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

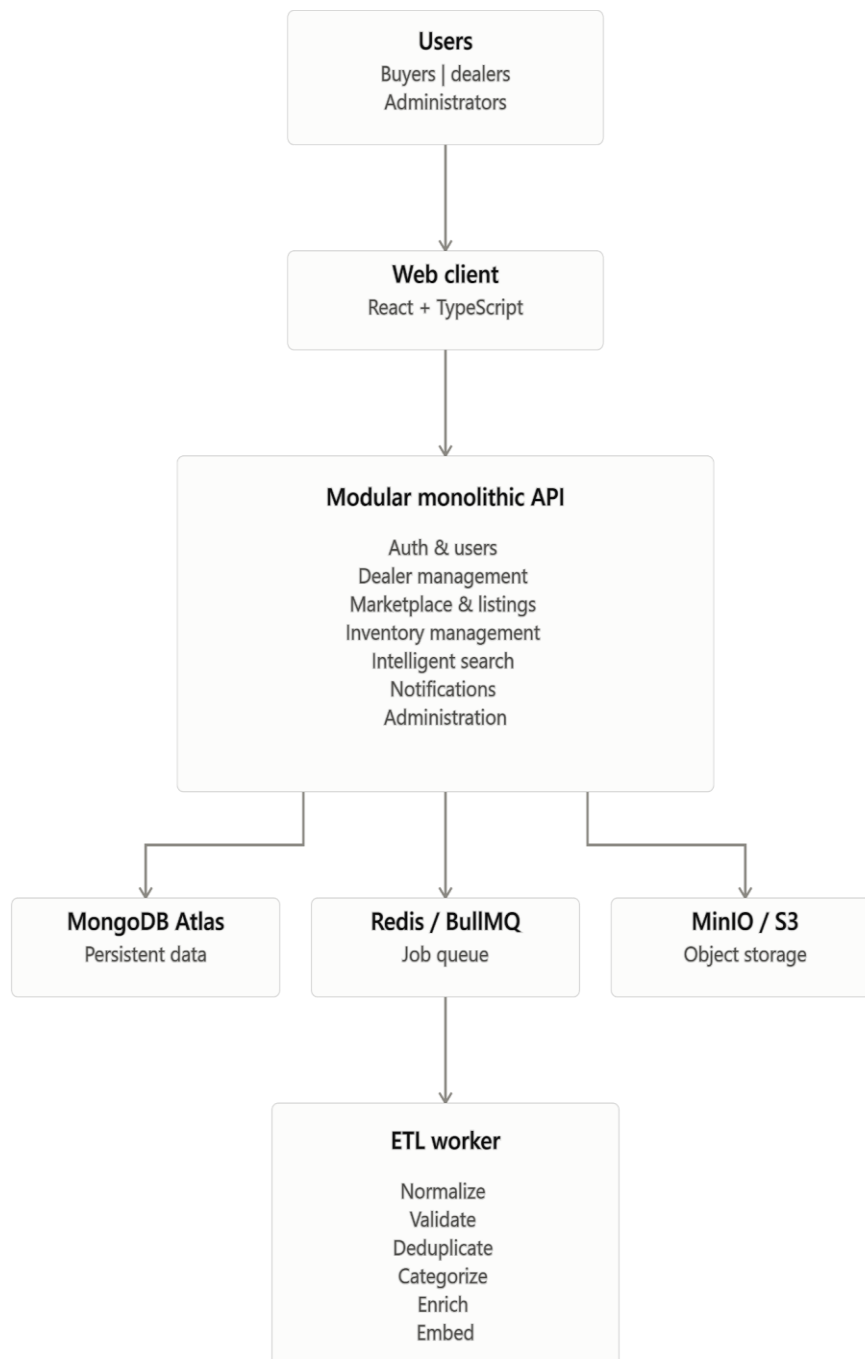


Figure 1: High level Architecture

Figure 1 presents the high-level architecture and system boundary of MotorX. The main client application is the React/TypeScript web application, used by buyers to browse and search, by dealers to manage inventory, and by administrators to configure and monitor the platform. This application

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

communicates with the Core Backend API, which coordinates the platform's dealer, marketplace, inventory, search, notification, and administration functions.

The backend communicates with the separately deployable ETL Worker to perform normalization, deduplication, categorization, enrichment and embedding generation for uploaded inventory. MongoDB Atlas is used for permanent system data, Redis supports caching, short-lived state, rate-limiting counters and background-job queues, and object storage (S3/MinIO) holds original CSV uploads and vehicle images. External services provide identity/authentication (Firebase) and semantic-embedding generation (Hugging Face model endpoint).

The diagram also emphasizes that the core backend is deployed as one application containing separate service modules (a modular monolith), while the ETL Worker can be deployed and scaled independently due to its greater and burstier processing requirements.

3. Architectural Goals and Constraints

The following requirements from the SRS have a significant impact on the architecture (traced identifiers refer to the SRS requirement subsets):

Architectural Goals

- Responsiveness under load: large inventory uploads must not block normal API traffic (FR-ETL-01/02, PSR-06) — addressed by separating ETL processing into an asynchronous worker.
- Data consistency and quality: dealer-submitted data in inconsistent formats must be normalized and deduplicated automatically without manual approval (FR-ETL-07 to FR-ETL-22).
- Search quality and speed: buyers must be able to search using free-text natural language as well as structured filters, with results ranked by relevance (FR-SEARCH-08 to FR-SEARCH-20, PSR-02/03).
- Portability: the system must run identically in a local Docker environment and on AWS (project proposal, Section 4; DC-03/DC-04).
- Scalability: the architecture must support at least 1,000 dealer accounts, 100,000 listings and 10,000 concurrent users as a long-term target (PSR-07).
- Security: role-based access control and Firebase-token-based authentication must be enforced on every protected operation (FR-USER-04 to FR-USER-11, PSR-08 to PSR-16).

Architectural Constraints

- DC-01: The primary application shall use a modular monolithic backend with logical modules for Auth & Users, Dealers, Marketplace, Inventory, Search, Notifications and Administration.
- DC-02: ETL processing shall be implemented as a separate asynchronous worker using Redis/BullMQ.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

- DC-03: The technology stack shall use React, TypeScript, Node.js, Express.js, MongoDB, Redis, BullMQ and Docker (or approved alternatives).
- DC-04: Original inventory files shall be stored in object storage (S3/MinIO) rather than in the primary database.
- The architecture shall avoid unnecessary distributed microservice infrastructure and remain simple enough for a three-person development team to implement and operate (SRS Section 2.5).
- All technologies used shall be free/open-source or available on a free tier (project proposal, Section 6).

4. Use-Case View

The Use-Case View describes the main interactions between MotorX and its users. It identifies the primary system actors, their main use cases, and representative end-to-end workflows.

4.1 Actors

MotorX has three primary actors:

Actor	Description
Buyer	A user who browses and searches for vehicles and views vehicle and dealer information.
Dealer	An approved business user who manages vehicle listings and uploads vehicle inventory.
Administrator	An authorized user who manages users and dealers and monitors listings, inventory processing, and system activities.

External services such as Firebase Authentication, object storage, the embedding model, and email services support these interactions but are not primary business actors.

4.2 Actor Use Cases

4.2.1 Buyer Use Cases

Register and Login

Allows a buyer to register and authenticate using Firebase Authentication.

View and Update Profile

Allows an authenticated buyer to view and update supported profile information.

Browse Vehicles

Displays available vehicle listings in the marketplace.

Search Using Structured Filters

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Allows the buyer to search using filters such as make, model, price range, year, mileage, fuel type, transmission, category, and location.

Search Using Natural Language

Allows the buyer to enter queries such as “affordable automatic SUV under 2 million near Colombo.” The system interprets the query and uses structured filtering, fuzzy matching, and semantic similarity to find relevant listings.

View Vehicle Details

Displays detailed information about a selected vehicle, including its attributes, images, price, availability, and description.

View Dealer Information

Displays relevant information about the dealer associated with a vehicle listing.

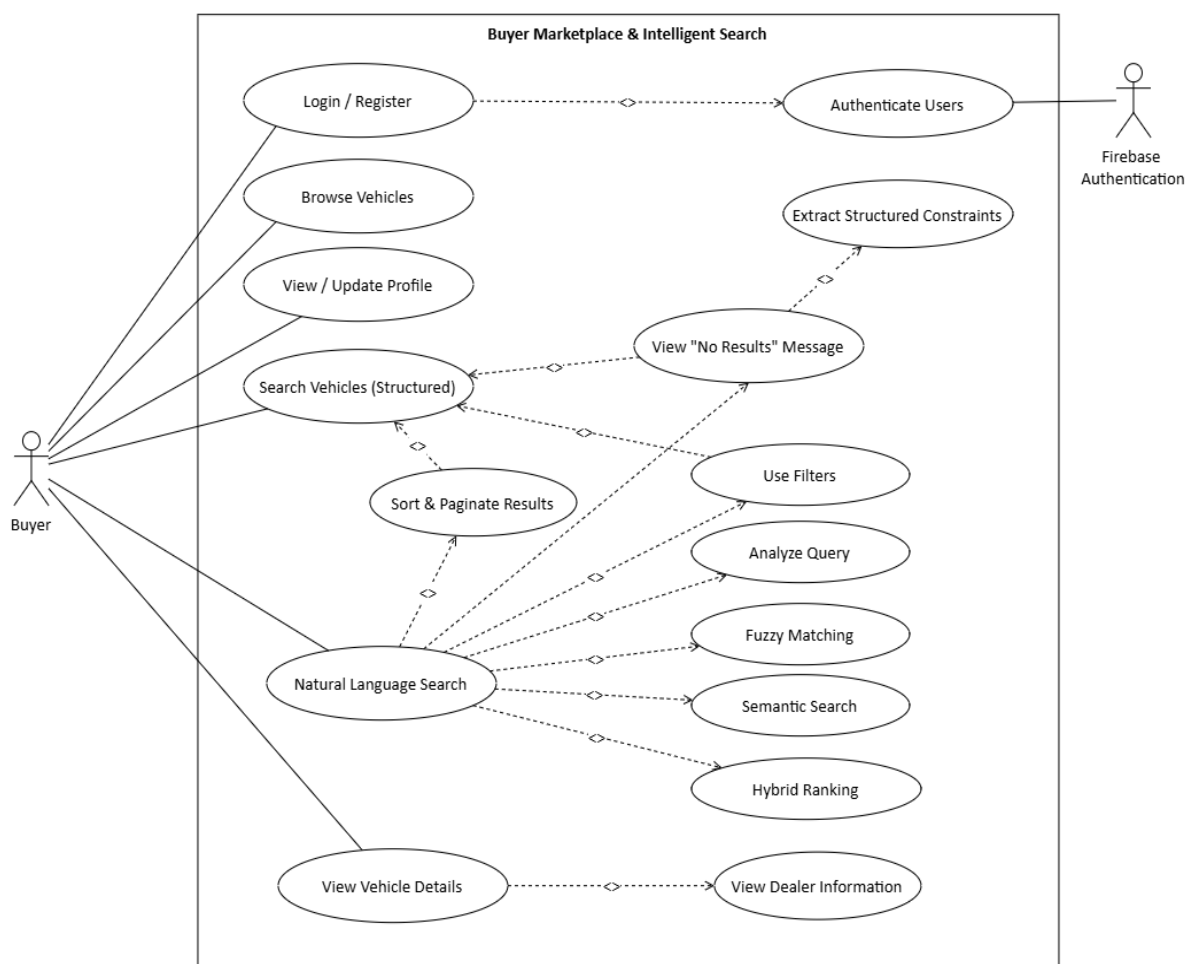


Figure 2: Buyer Use Case Diagram

4.2.2 Dealer Use Cases

Register and Login

Allows a dealer user to register and authenticate using Firebase Authentication.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Request Dealer Account Activation

Allows a registered user to request approval for a dealer account.

Manage Dealer Profile

Allows an approved dealer to maintain business information and contact details.

View Dealer Dashboard

Displays the dealer's listings, inventory information, upload history, processing statistics, and related activities.

Create Vehicle Listing

Allows an approved dealer to create an individual vehicle listing.

Update Vehicle Listing

Allows a dealer to modify listings belonging to their dealer account.

Manage Listing Availability

Allows a dealer to change whether a vehicle listing is currently available.

Upload and Manage Vehicle Images

Allows a dealer to upload, replace, and remove images associated with their vehicle listings. Image files are stored in object storage while image metadata is stored within the Listing document.

Upload Inventory CSV

Allows a dealer to upload a supported CSV inventory file for asynchronous processing.

Monitor Upload Status

Allows a dealer to view the current status of an inventory upload job.

View Processing Summary

Displays the number of successfully processed, rejected, and duplicate records.

Review Rejected Records

Allows the dealer to view records that failed validation and the reasons for rejection.

Correct and Re-upload Invalid Records

Allows a dealer to correct invalid inventory data and submit it again for processing.

View In-Application Notifications

Displays inventory-processing and other relevant dealer notifications within MotorX.

Receive Email Notifications

Allows important system notifications, such as inventory-processing results, to also be delivered to the dealer by email where applicable.



Figure 3: Dealer Use Case Diagram

4.2.3 Administrator Use Cases

Login

Allows an administrator to authenticate before accessing administrative functionality.

Manage User Accounts

Allows administrators to view and manage registered user accounts.

Approve or Reject Dealer Requests

Allows administrators to review dealer account requests and approve or reject them.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Manage Dealer Accounts

Allows administrators to view and manage dealer accounts and their account status.

Monitor Vehicle Listings

Allows administrators to monitor vehicle listings available through the marketplace.

Monitor Inventory Processing

Allows administrators to view inventory upload jobs and processing activities.

View System Health

Displays the operational status of critical system components, including the backend application, MongoDB, Redis/BullMQ, ETL Worker, and object storage where applicable.

View System-Level Statistics

Displays information such as the total number of vehicle listings and registered dealers.

View Recent System Activities

Displays relevant recent activities, including inventory uploads and processing failures.

Review Audit Information

Allows administrators to review recorded information about significant administrative and system activities.

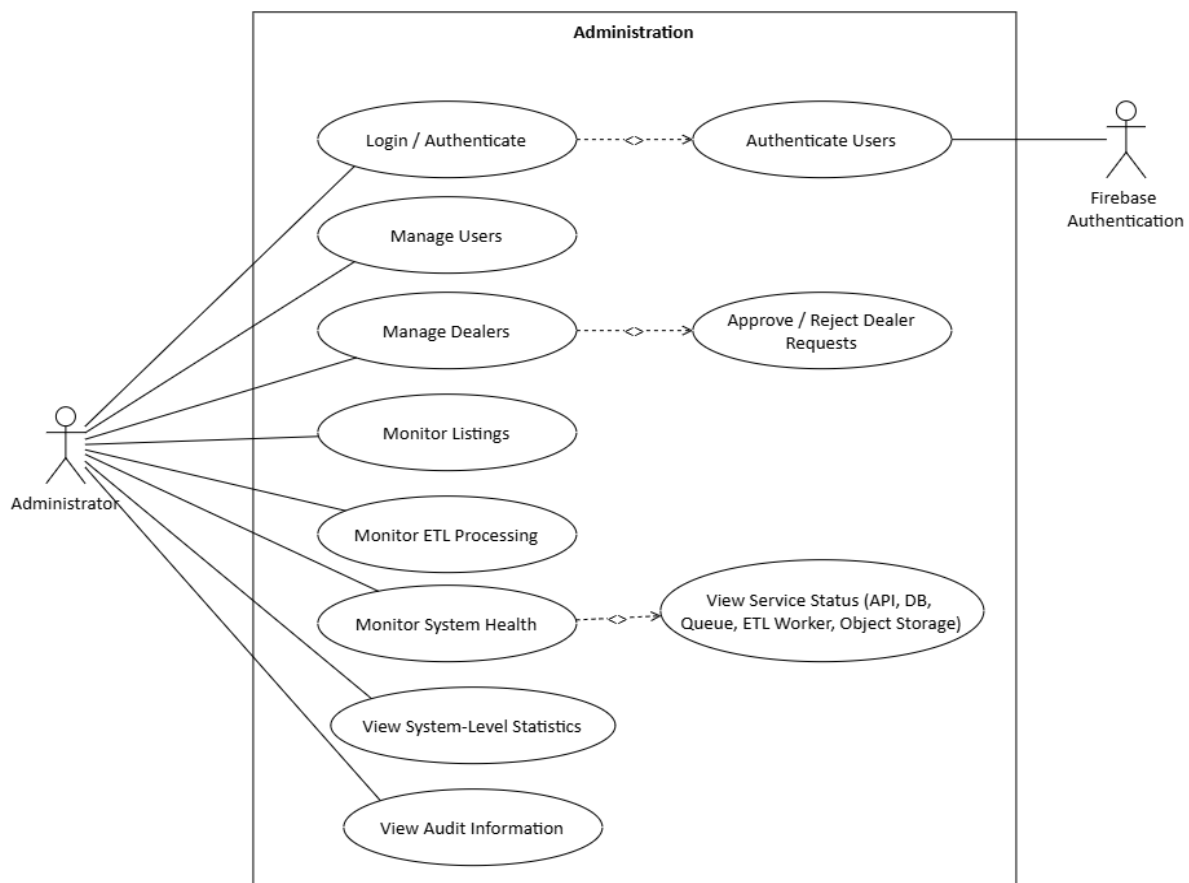


Figure 4: Administrator Use Case Diagram

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

4.3 Use-Case Realizations

The following use-case realizations describe representative end-to-end interactions between users and the major architectural components of MotorX.

4.3.1 User Authentication

Primary Actor: Buyer, Dealer, or Administrator

Precondition: The user has registered or has valid authentication credentials.

Main Flow

1. The user enters their authentication information through the React web client.
2. The web client sends the authentication request to Firebase Authentication.
3. Firebase Authentication verifies the user's identity.
4. After successful authentication, Firebase returns a Firebase ID token to the web client.
5. The web client includes the token in protected API requests.
6. The Express backend verifies the token using Firebase Admin SDK.
7. The backend identifies the corresponding MotorX user.
8. The backend checks the user's application role.
9. The requested functionality is allowed or denied according to the user's authorization.

Postcondition: The authenticated user can access functionality permitted by their assigned role.

Alternative Flow — Authentication Failure: *If Firebase Authentication rejects the credentials or the token cannot be verified, access is denied and an appropriate authentication error is displayed.*

Contributing design elements: React Web Client, Firebase Authentication, Authentication Middleware, User Module, MongoDB Atlas.

4.3.2 Dealer Uploads Vehicle Inventory

Primary Actor: Dealer

Preconditions:

- The dealer is authenticated.
- The dealer account has been approved.
- The uploaded file uses the supported CSV format.

Main Flow

1. The dealer selects an inventory CSV file from the Dealer Dashboard.
2. The frontend sends the file to the backend.
3. The Inventory module validates the file type and basic upload requirements.
4. The original CSV file is stored in object storage.
5. The backend creates an Upload Job in MongoDB with a Pending status.
6. A background processing job is added to Redis/BullMQ.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

7. The ETL Worker retrieves the queued job.
8. The ETL Worker reads the uploaded file from object storage.
9. Vehicle records are extracted and normalized.
10. Required fields, data types, and supported values are validated.
11. Duplicate records are identified.
12. Valid vehicle records are categorized and enriched.
13. Searchable vehicle information is prepared and semantic embeddings are generated using the configured embedding model.
14. Valid records are stored as Listings in MongoDB.
15. Invalid records are stored as Rejected Records with their rejection reasons.
16. The Upload Job is updated with its final status and processing statistics.
17. The system creates an in-application notification for the dealer and may also send an email notification.
18. The Dealer Dashboard displays the processing result.

Postcondition: Valid vehicle records are available as listings and rejected records are available for dealer review.

Alternative Flow — Invalid CSV: If the uploaded file is unsupported or does not meet the required CSV format, the system rejects the upload and displays an appropriate error.

Alternative Flow — Partial Success: If only some records are invalid, valid records are imported while invalid records are stored separately as Rejected Records.

Alternative Flow — Processing Failure: If the background processing job fails, the failure is recorded, the Upload Job is updated appropriately, and the dealer is notified.

Contributing design elements: Dealer Web Client, Inventory Module, Object Storage, Redis/BullMQ, ETL Worker, Embedding Model, MongoDB Atlas, Notification Module.

4.3.3 Buyer Searches Using Natural Language

Primary Actor: Buyer

Precondition: Searchable vehicle listings are available.

Main Flow

1. The buyer enters a natural-language query in the search interface.
2. The web client sends the search request to the Express backend.
3. The Query Analyzer analyzes the query.
4. Recognizable structured constraints are extracted from the query.
5. Remaining unstructured search intent is identified.
6. The Search Module applies the extracted structured constraints to obtain suitable candidate listings.
7. Fuzzy matching is used where appropriate to handle supported spelling variations and typographical errors.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

8. A query embedding is generated using the compatible embedding model.
9. Semantic similarity is calculated between the query and relevant listing embeddings.
10. The Search Module combines structured constraint satisfaction, fuzzy relevance, and semantic similarity to rank the results.
11. Sorting and pagination are applied.
12. The backend returns the ranked results to the web client.
13. The buyer views the search results and may open a vehicle listing.

Postcondition: The buyer receives vehicle listings relevant to the natural-language query.

Alternative Flow — No Results: *If no suitable listings are found, the system displays a message indicating that no vehicles were found and allows the buyer to modify the query or filters.*

Contributing design elements: Buyer Web Client, App Server, Query Analyzer, Search Module, Embedding Model, MongoDB Atlas.

4.3.4 Buyer Searches Using Structured Filters

Primary Actor: Buyer

Precondition: Vehicle listings are available.

Main Flow

1. The buyer opens the marketplace search interface.
2. The buyer selects one or more structured filters.
3. The web client sends the selected filters to the backend.
4. The Search Module validates and applies the filters.
5. MongoDB returns listings satisfying the selected conditions.
6. The Search Module applies the requested sorting and pagination.
7. The backend returns the matching listings.
8. The web client displays the results to the buyer.

Postcondition: The buyer receives listings matching the selected structured filters.

Alternative Flow — No Results: *If no listings satisfy the filters, the system displays a “No vehicles found” message.*

Contributing design elements: Buyer Web Client, Search Module, MongoDB Atlas.

4.3.5 Administrator Approves Dealer Registration

Primary Actor: Administrator

Preconditions:

- The administrator is authenticated.
- A dealer account request exists.

Main Flow

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

1. The administrator opens the dealer approval interface.
2. The backend retrieves pending dealer requests from MongoDB.
3. The system displays the requests to the administrator.
4. The administrator selects a dealer request.
5. The administrator reviews the submitted dealer information.
6. The administrator approves or rejects the request.
7. If approved, the dealer account status is updated, and the Dealer role is assigned to the corresponding MotorX user.
8. If rejected, the request status is updated accordingly.
9. The system records relevant audit information.
10. The system creates an in-application notification and may also send an email notification informing the user of the outcome.

Postcondition: The dealer request has been approved or rejected, and its status has been recorded.

Contributing design elements: Administrator Web Client, Administration Module, Dealer Module, User Module, Notification Module, MongoDB Atlas.

4.3.6 Dealer Manages Vehicle Listing and Images

Primary Actor: Dealer

Preconditions:

- The dealer is authenticated.
- The dealer account has been approved.

Main Flow

1. The dealer opens the listing management interface.
2. The dealer creates a new listing or selects an existing listing.
3. The dealer enters or updates vehicle information.
4. The dealer may upload vehicle images.
5. The Marketplace module validates the listing information and uploaded image files.
6. Valid image files are stored in object storage.
7. Image metadata and object-storage references are stored within the corresponding Listing document.
8. The Listing document is created or updated in MongoDB.
9. The updated listing information is displayed to the dealer.

Postcondition: The vehicle listing and its associated image information are stored and available according to the listing's availability status.

Alternative Flow — Invalid Listing Data: If required listing information is missing or invalid, the system displays validation errors and does not save the invalid changes.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Alternative Flow — Invalid Image: *If an uploaded image has an unsupported file type or exceeds the allowed file size, the image is rejected and an appropriate error is displayed.*

Contributing design elements: Dealer Web Client, Marketplace Module, Object Storage, MongoDB Atlas.

4.3.7 Administrator Monitors the System

Primary Actor: Administrator

Precondition: The administrator is authenticated and authorized to access administrative functionality.

Main Flow

1. The administrator opens the Administration Dashboard.
2. The system retrieves system-level statistics such as the total number of vehicle listings and registered dealers.
3. The system retrieves the operational status of critical application components.
4. The system displays the status of the backend application, MongoDB, Redis/BullMQ, ETL Worker, and object storage where applicable.
5. The system retrieves recent inventory uploads and processing failures.
6. The dashboard displays the collected system information.
7. The administrator may review recent activities and relevant audit information.

Postcondition: The administrator has an overview of current MotorX system operations and recent processing activity.

Contributing design elements: Administrator Web Client, Administration Module, MongoDB Atlas, Redis/BullMQ, ETL Worker.

5. Logical View

The Logical View describes how the system is divided into functional modules, packages, and domain classes. It focuses on the responsibilities of each part of the system and the important relationships between them rather than their physical deployment.

The backend is organized as a single deployable modular monolith. Each package owns a clearly bounded responsibility and communicates with other packages only through defined interfaces. The ETL pipeline is logically part of this same design but executes inside a separately deployed ETL Worker process, since inventory processing is bursty and benefits from independent scaling.

5.1 Overview

a package diagram showing the seven business packages below, the separately deployed ETL Worker, and the four supporting packages they depend on.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

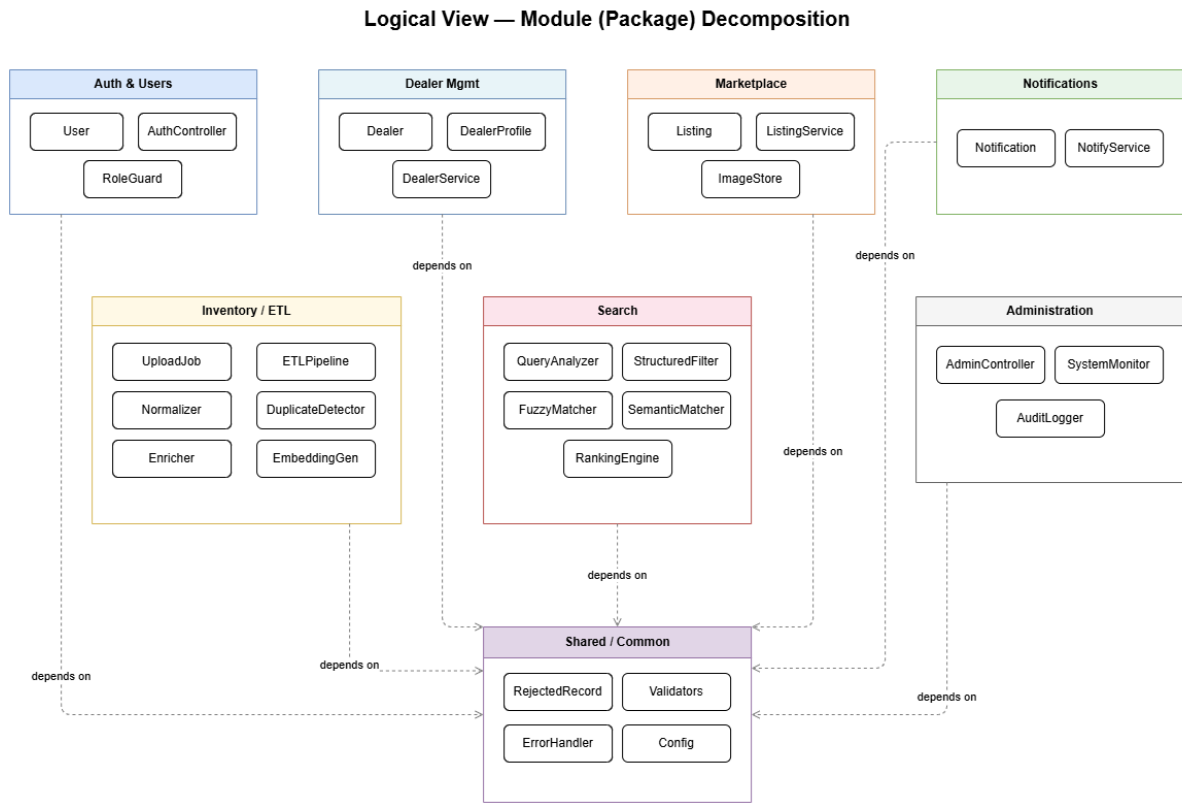


Figure 5: Logical Service Decomposition Diagram

The platform decomposes into seven business packages that make up the core backend, plus the separately deployable ETL Worker:

- **Identity & Access** — authenticates users through Firebase and authorizes every protected operation.
- **Dealer Management** — owns dealer profiles, the dealer-approval workflow, and dashboard statistics.
- **Marketplace** — owns vehicle-listing data and images, and serves buyer-facing browse/detail reads.
- **Inventory / ETL** — executed by the ETL Worker; turns raw dealer uploads into standardized, searchable listings.
- **Search** — performs hybrid intelligent search combining structured filtering, fuzzy matching, and semantic similarity.
- **Notifications** — distributes processing-outcome alerts to dealers, in-application and by email.
- **Administration** — backed by the Audit module, gives administrators oversight of accounts, listings, and system health.

Four supporting packages — Database, Background Tasks, Integrations, and Common Configuration — provide shared infrastructure that the business packages depend on but do not own domain logic themselves (detailed in 5.2.8)

5.2 Architecturally Significant Design Packages

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

5.2.1 Identity & Access Package

Responsibility: Verifies Firebase ID tokens, validates roles and permissions, and resolves the authenticated user's profile for every protected operation.

Class	Key attributes	Key operations	Responsibility
UserAccount	firebaseUid, email, role, accountStatus, createdAt	verifyToken()	The common authentication account shared by all users; holds the Firebase identity, assigned role, and account status.
Buyer	preferences, searchHistory	—	Role-specific profile for a buyer, associated with a UserAccount.
Dealer	businessName, registrationNo, approvalStatus	requestActivation()	Role-specific profile for a dealer; tracks approval status ahead of gaining dealer functionality.
Administrator	adminLevel	approveDealer()	Role-specific profile for an administrator, with elevated system-management privileges.

Key relationship: a UserAccount is associated with exactly one of Buyer, Dealer, or Administrator, depending on its assigned role.

5.2.2 Dealer Management Package

Responsibility: Maintains dealer business information and runs the dealer-account approval workflow.

Class	Key attributes	Key operations	Responsibility
DealerProfile	address, phone, location, description, contactEmail	updateProfile()	Business identity of an approved dealer — contact and location details shown to buyers and used for dashboard aggregation.

Key relationship: a Dealer (Identity & Access) owns exactly one DealerProfile once approved; DealerProfile in turn owns the dealer's Listings (5.2.3) and UploadJobs (5.2.4).

5.2.3 Marketplace Package

Responsibility: Owns vehicle-listing data and images, enforces dealer ownership on writes, and serves buyer-facing browse and detail reads.

Class	Key attributes	Key operations	Responsibility
Listing	listingId, make, model, year, price, mileage, fuelType, transmission, category, availabilityStatus	updateStatus()	A vehicle record shown in the marketplace; created either directly by a dealer or by the ETL pipeline.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

ListingImage	imageId, objectStorageUrl, isPrimary	—	A single image associated with a Listing; the file itself is held in object storage (MinIO in local development, AWS S3 in production), with only the reference stored here.
--------------	--------------------------------------	---	--

Key relationship: a Dealer owns zero or more Listings; each Listing has zero or more ListingImages.

5.2.4 Inventory / ETL Package

Responsibility: The most architecturally significant package in the system. Implements the asynchronous ETL pipeline — extraction, normalization, validation, duplicate detection, categorization, enrichment, and embedding generation — that turns a raw CSV upload into standardized, searchable Listings. Executed by the separately deployed ETL Worker, which consumes queued jobs from Redis/BullMQ and reads the source file from object storage.

Class	Key attributes	Key operations	Responsibility
UploadJob	jobId, status, totalRecords, processedRecords, successfulRecords, rejectedRecords	updateProgress()	Tracks one dealer file's processing lifecycle — Pending / Processing / Completed / Completed with Errors / Failed — and its running statistics.
RejectedRecord	rowNumber, rawData, rejectionReason	—	A single inventory row that failed validation, retained with enough of the original data to identify and correct it.

Key relationship: an UploadJob belongs to one Dealer and produces zero or more Listings and zero or more RejectedRecords.

5.2.5 Search Package

Responsibility: Implements hybrid intelligent search — structured filtering, fuzzy matching, and semantic similarity — and ranks results by combined relevance.

Class	Key attributes	Key operations	Responsibility
ListingEmbedding	embeddingId, vectorValues, modelVersion	—	The semantic vector embedding generated for a Listing's searchable text, used for similarity matching.
QueryAnalyzer (<i>class utility</i>)	—	extractConstraints()	Parses a natural-language query into structured constraints and residual free-text intent.
RankingEngine (<i>class utility</i>)	—	rank()	Combines structured constraint satisfaction, fuzzy relevance, and semantic similarity into a single ordered result set.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Key relationship: each Listing that has completed ETL processing is associated with exactly one ListingEmbedding, generated using the same embedding model version as query-time embeddings, so the two remain comparable.

5.2.6 Notifications Package

Responsibility: Delivers system-generated alerts to the correct recipient, in-application and by email, without delaying the operation that triggered them.

Class	Key attributes	Key operations	Responsibility
Notification	recipientUserId, type, message, isRead, emailStatus	markAsRead()	A single alert — upload completion/failure or dealer-approval outcome — scoped to one recipient.

Key relationship: a Notification belongs to exactly one recipient UserAccount. Delivery, including email dispatch, is handled by the Background Tasks package (§5.2.8) so it never blocks the triggering request.

5.2.7 Administration Package

Responsibility: Gives administrators oversight of accounts, listings, and system health, and records an audit trail of significant administrative actions.

Class	Key attributes	Key operations	Responsibility
AuditLogEntry	actorUserId, eventType, action, resourceType, outcome	—	An immutable record of a significant administrative or security-relevant action, such as a dealer approval decision.

Key relationship: an AuditLogEntry references the acting Administrator/UserAccount and the resource it affects (for example, a Dealer or UserAccount).

5.2.8 Supporting Packages

These packages provide shared infrastructure to the business packages above but hold no domain logic of their own.

Supporting Package	Class utilities	Responsibility
Database	RepositoryBase, ConnectionManager	Database connection management, migrations/indexes, and shared persistence utilities used by every repository.
Background Tasks	NotificationDispatcher	Executes notification delivery (in-application and email) and other asynchronous, non-ETL operations via Redis/BullMQ.

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

Integrations	ObjectStorageClient, EmbeddingModelClient, FirebaseAdminClient	Wraps communication with object storage (MinIO/S3), the embedding-model endpoint, and Firebase Authentication.
Common Configuration	EnvConfig, Validators, ErrorHandler	Environment configuration, logging, shared constants, and reusable validators.

Business logic must remain inside the relevant business package; these utilities are not permitted to hold domain rules of their own.

5.3 Core Domain Model

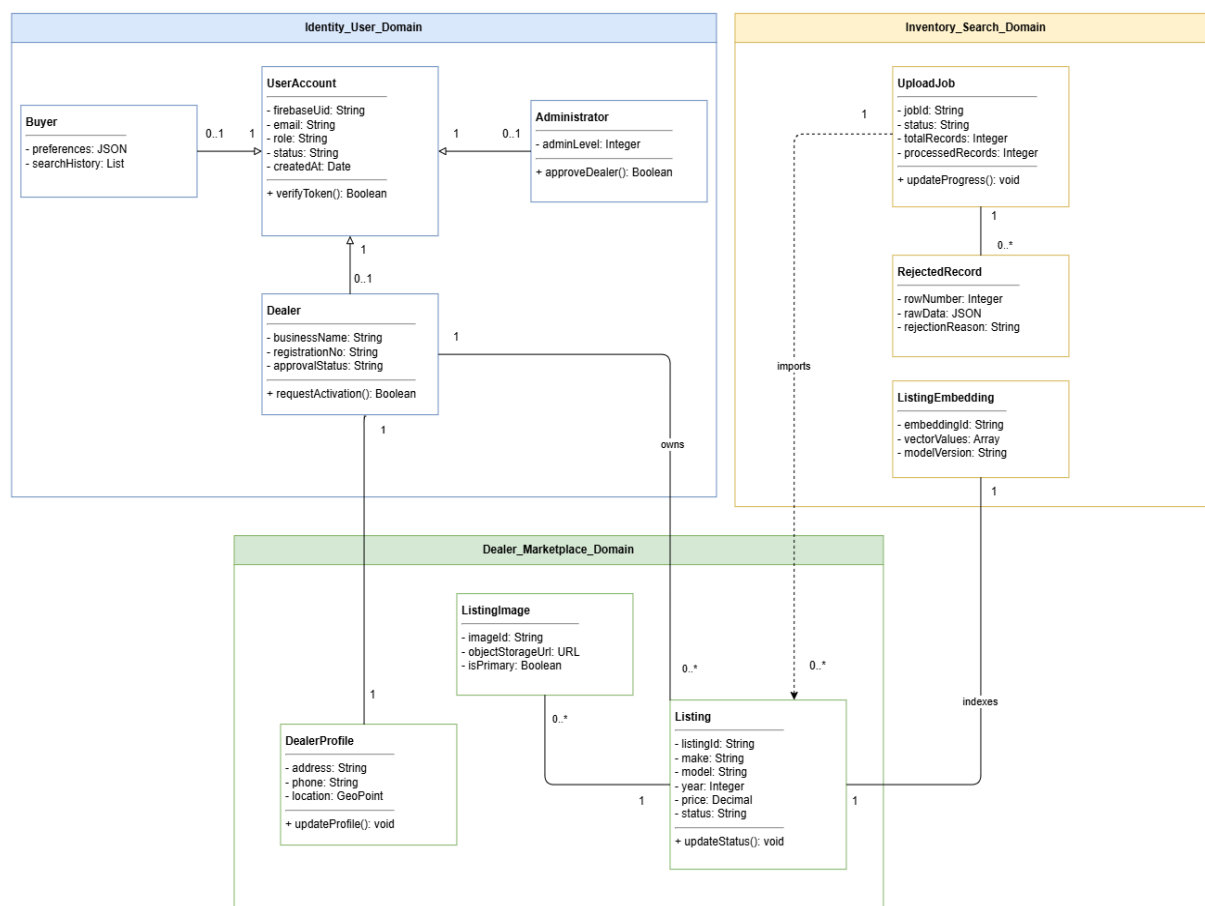


Figure 6: Core Domain Class Diagram

Figure 6 presents the architecturally significant domain classes and their relationships, grouped into the Identity & User Domain, the Dealer/Marketplace Domain, and the Inventory/Search Domain.

5.3.1 Identity & User Domain

Class	Responsibility
-------	----------------

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

UserAccount	Represents the common authentication account, Firebase UID, account status, and assigned role.
Buyer	Represents a buyer's profile and preferences.
Dealer	Represents a dealer's business identity, linked to an approved UserAccount.
Administrator	Represents an administrative user with system-management privileges.

Buyer, Dealer, and Administrator are role-specific profiles associated with a UserAccount.

5.3.2 Dealer / Marketplace Domain

Class	Responsibility
DealerProfile	Business name, contact information, address/location, description.
Listing	Represents a vehicle listing: attributes, availability, description, image references.
ListingImage	Represents an individual image associated with a listing, stored in Object Storage.

A Dealer owns zero or more Listings. Each Listing may have zero or more ListingImages.

5.3.3 Inventory / Search Domain

Class	Responsibility
UploadJob	Tracks a dealer file's processing lifecycle and statistics.
RejectedRecord	Represents a single inventory row that failed validation, with its reason.
ListingEmbedding	Represents the semantic vector embedding generated for a Listing, used by the Search module.

An UploadJob produces zero or more Listings and zero or more RejectedRecords. Each Listing that has completed ETL processing is associated with exactly one ListingEmbedding.

5.4 Important Domain Relationships

The most important logical relationships are:

A UserAccount is associated with a Buyer, Dealer, or Administrator profile.

A Dealer owns one or more Listings.

A Listing has zero or more ListingImages and exactly one ListingEmbedding once processed.

An UploadJob belongs to one Dealer and produces zero or more Listings and zero or more RejectedRecords.

A Notification belongs to exactly one recipient UserAccount.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

An AuditLogEntry references the acting Administrator/UserAccount and the affected resource (e.g., a Dealer or UserAccount).

These relationships maintain the separation between user identity, dealer/marketplace data, inventory-processing evidence, and the final searchable listing.

6. Process View

6.1 Process Organisation and Communication

The main runtime processes are organized into the following groups:

Process Group	Main Participants	Communication
User authentication	Web client, Firebase Authentication, App Server, MongoDB	Synchronous API requests; token verification
Dealer bulk inventory upload	Dealer, web client, App Server, Object Storage, Redis/BullMQ	Synchronous request to create the job; asynchronous enqueue
ETL processing	ETL Worker, Redis/BullMQ, Object Storage, embedding-model endpoint, MongoDB	Queue-mediated job consumption; service request-response for embeddings
Buyer search (structured / NL)	Buyer, web client, App Server, Search module, MongoDB	Synchronous API requests with coordinated matching stages
Dealer-approval workflow	Administrator, web client, App Server, Audit module, MongoDB	Authorized synchronous commands and audit recording
Notification delivery	ETL Worker / App Server, background workers, MongoDB, web client	Asynchronous task execution; client polling/fetch
Administrative management	Administrator, web client, App Server, MongoDB	Authorized API calls and persistent data updates

6.2 Dealer Bulk Inventory Upload Activity

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

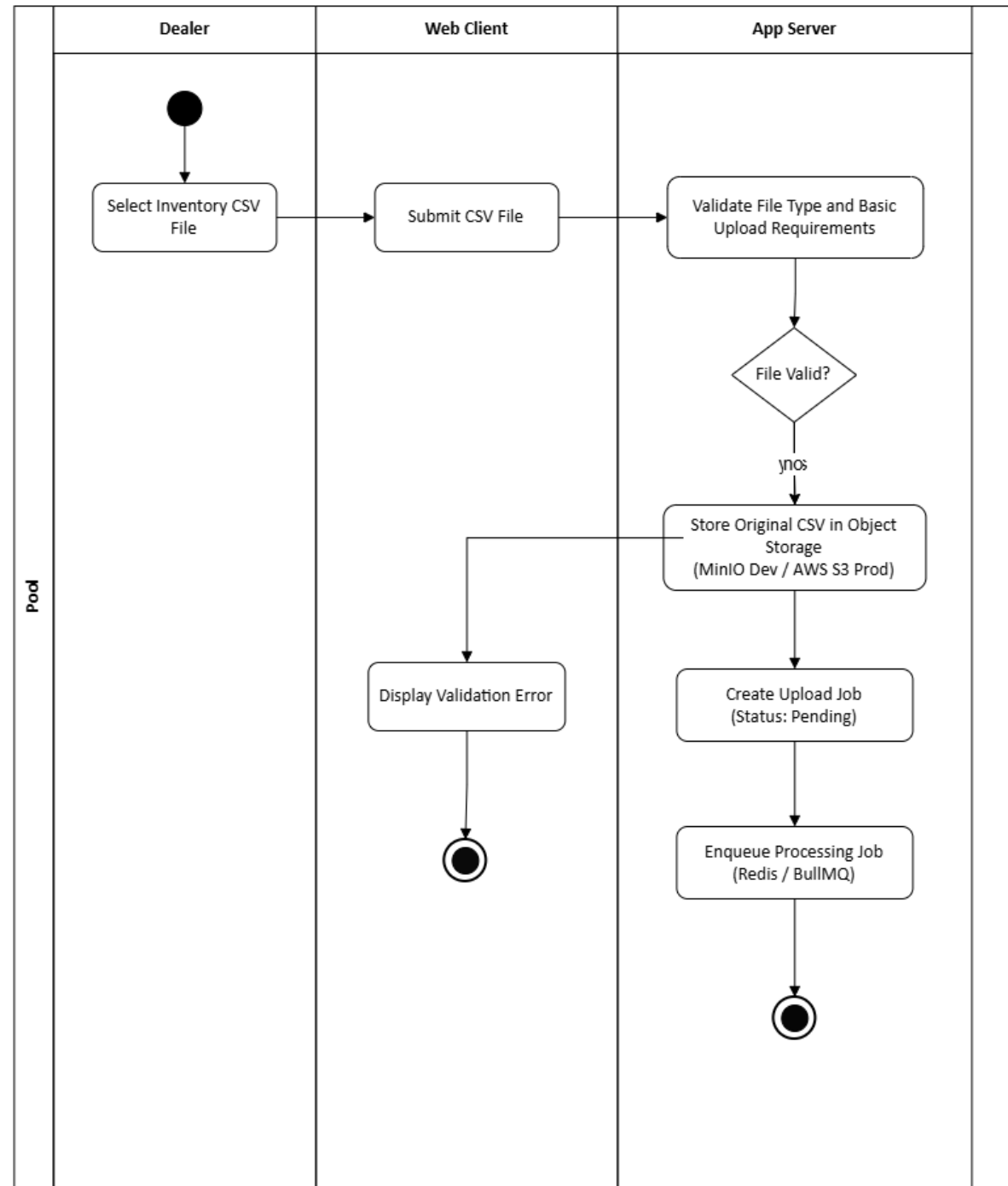


Figure 7: Dealer Bulk Inventory Upload Activity Diagram

Figure 7 presents the initial portion of the bulk-upload workflow, across the Dealer, Web Client, and App Server swimlanes. The dealer selects and submits a CSV file; the App Server validates the file type, stores the original file in Object Storage, creates an Upload Job in the Pending state, and enqueues a processing job on Redis/BullMQ. From this point, processing continues independently in the ETL Worker (Figure 8), so the dealer is not kept waiting for the full pipeline to complete.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

6.3 ETL Pipeline Processing Activity

then make those changes and give me the diagram. you have to recommend me good flow for first few steps. And the previous constrains are still required. need to be balck and white. need to be shorter for fir to the page.

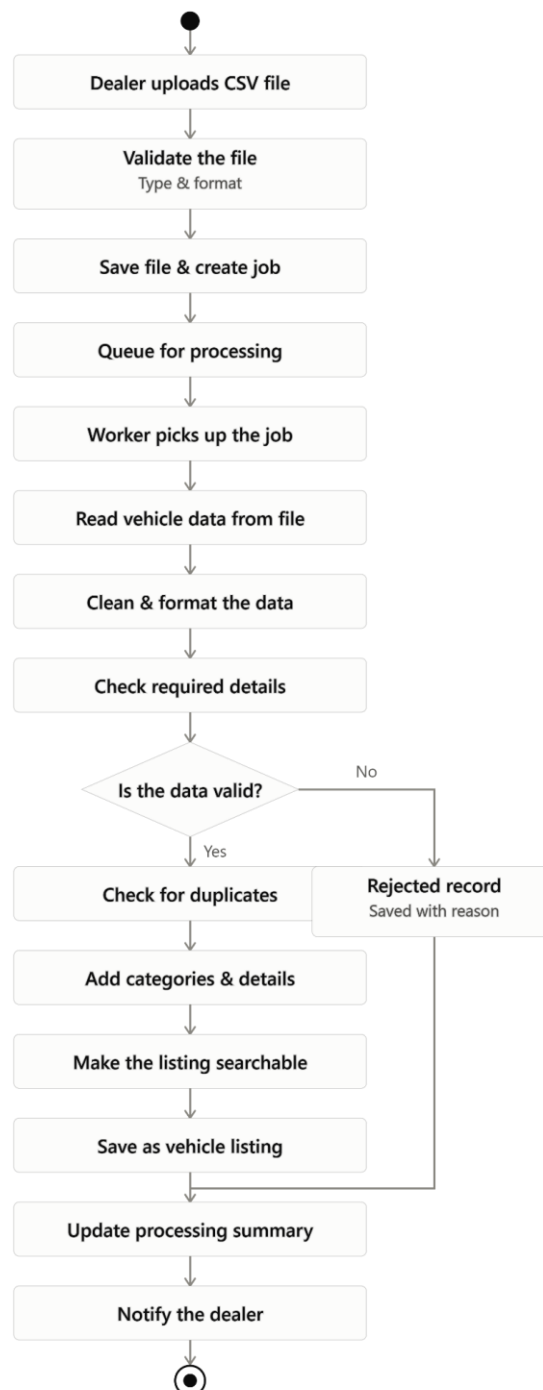


Figure 8: ETL Pipeline Processing Activity Diagram

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Figure 8 describes how the ETL Worker processes a queued Upload Job. Each extracted record is normalized and validated; valid records proceed through duplicate detection, categorization, enrichment, and embedding generation before being persisted as Listings, while invalid records are stored as Rejected Records with a specific reason. After all records are processed, the Upload Job is updated with a processing summary (successful, rejected, duplicate counts) and the dealer is notified.

6.4 Dealer Uploads Inventory Sequence

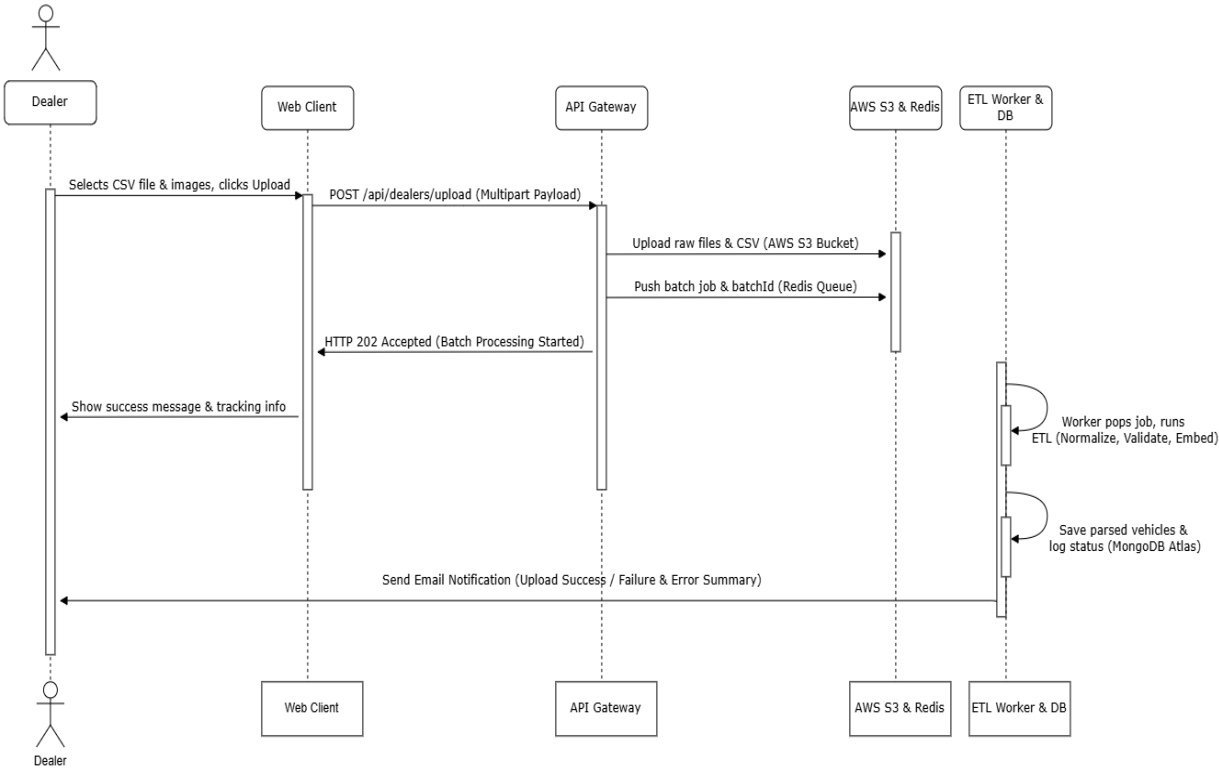


Figure 9: Dealer Uploads Inventory Sequence Diagram

Figure 9 shows the interaction among the Dealer, Web Client, App Server, Object Storage, Redis/BullMQ, and MongoDB during the synchronous portion of inventory upload — ending once the job has been durably queued and an acknowledgement returned to the dealer.

6.5 User Login Sequence

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

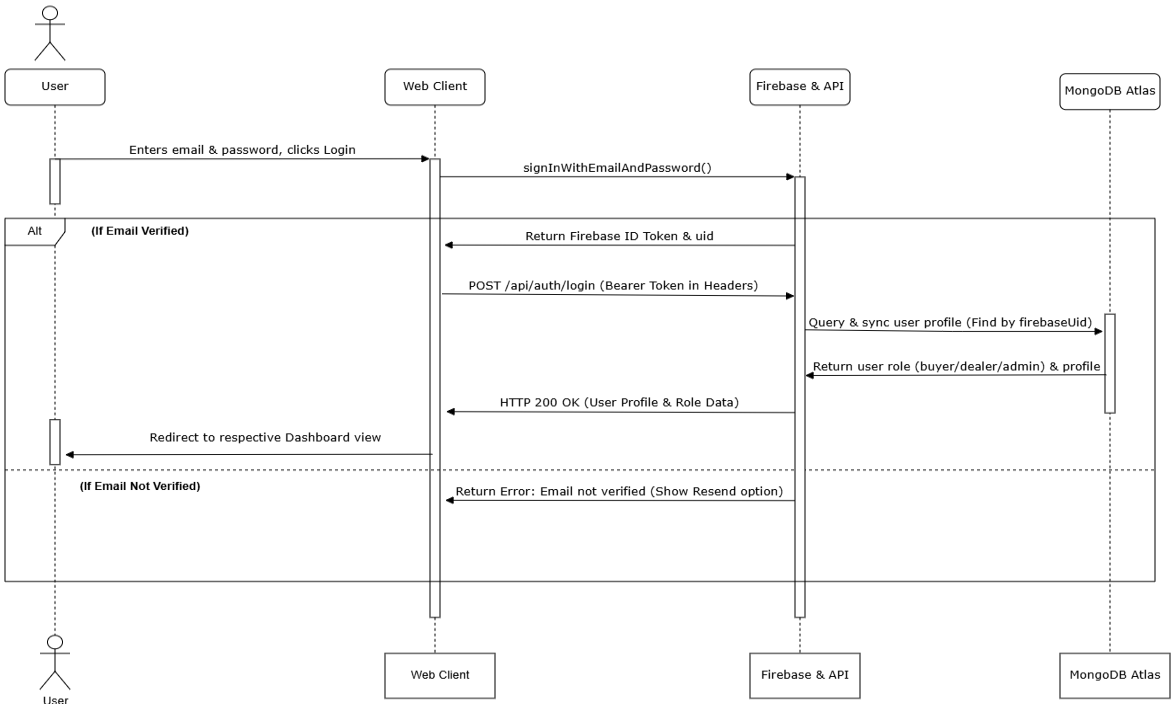


Figure 10: User Login Sequence Diagram

Figure 10 illustrates the login process involving the User, Web Client, Firebase Authentication, App Server, and MongoDB. Firebase performs credential verification and token issuance; the App Server is only responsible for verifying the token and resolving the application role, keeping password handling entirely outside the MotorX codebase.

6.6 Buyer Natural-Language Search Sequence

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

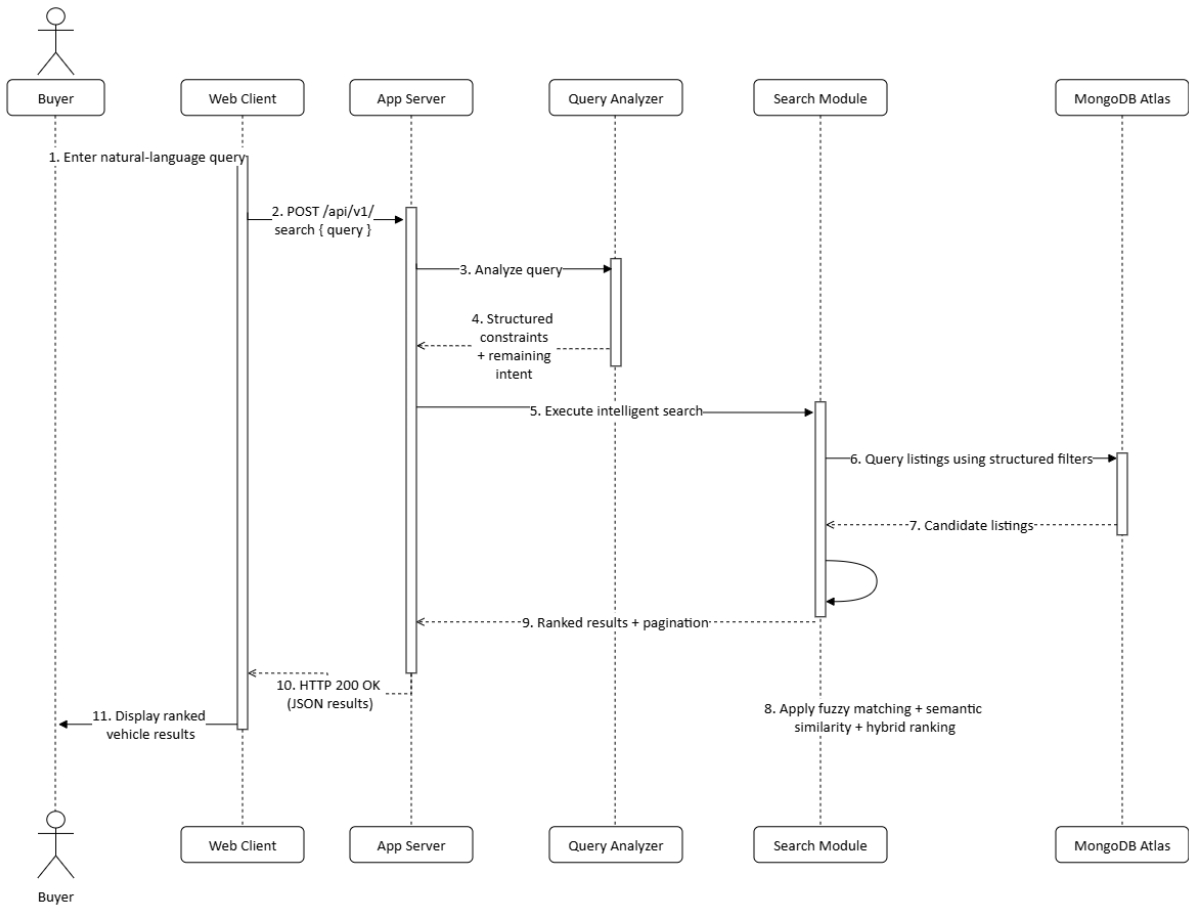


Figure 11: Buyer Natural-Language Search Sequence Diagram

Figure 11 shows the interactions among the Buyer, Web Client, App Server, Query Analyzer, Search Module, and MongoDB during a natural-language search request, ending with a hybrid-ranked result set returned to the buyer.

6.7 Buyer Structured Filter Search Sequence

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

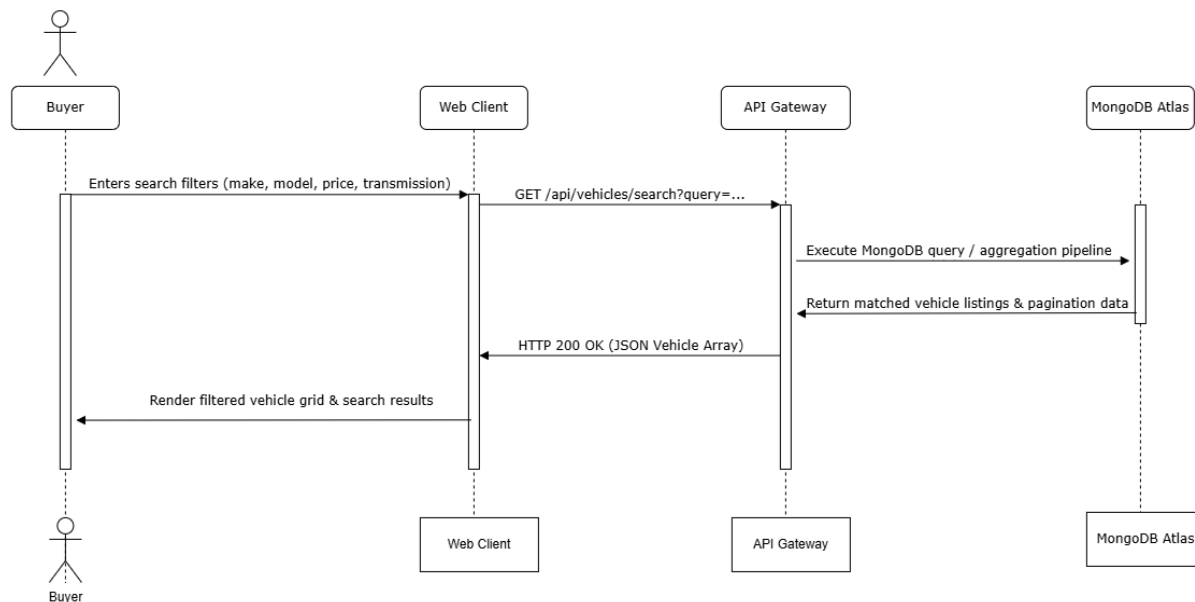


Figure 12: Buyer Structured Filter Search Sequence Diagram

Figure 12 shows the simpler, purely structured search path used when a buyer applies filters directly without a natural-language query, bypassing the Query Analyzer and fuzzy/semantic matching stages for a faster response.

6.8 Semantic Embedding Generation Sequence

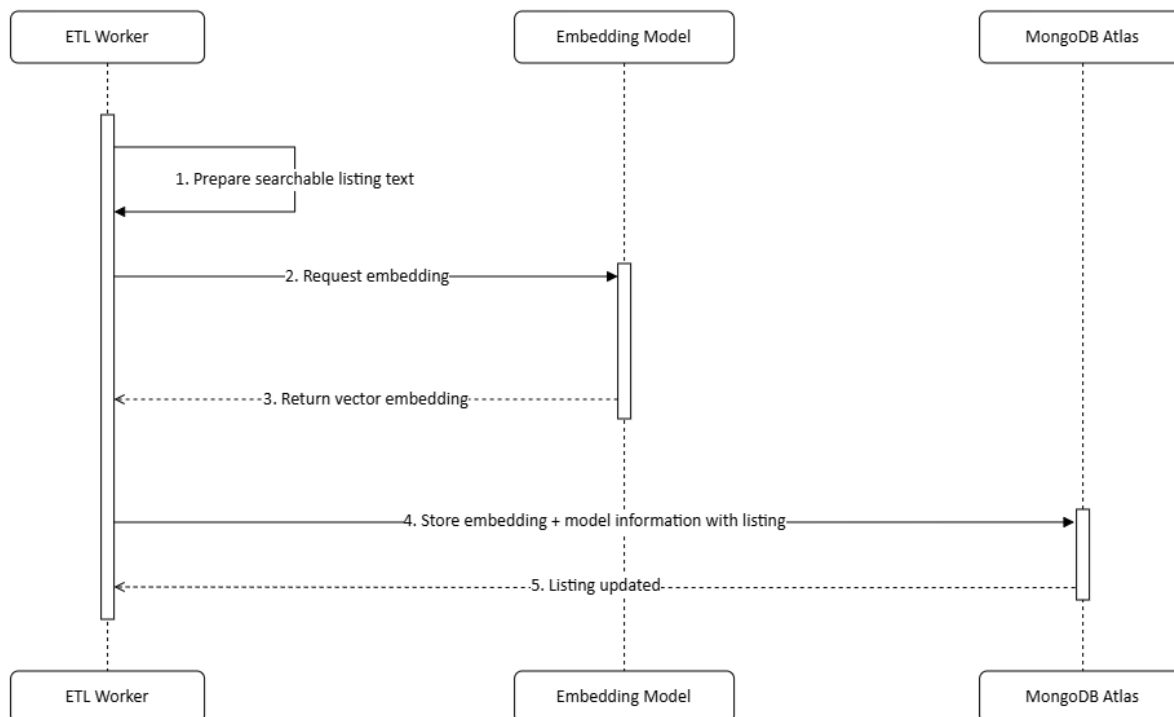


Figure 13: Semantic Embedding Generation Sequence Diagram

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

Figure 13 details the embedding-generation step that occurs during ETL processing (referenced from Figure 8) and is structurally identical to the embedding step used for buyer queries during natural-language search (Figure 11), ensuring listing and query embeddings remain comparable.

6.9 Administrator Approves Dealer Registration Sequence

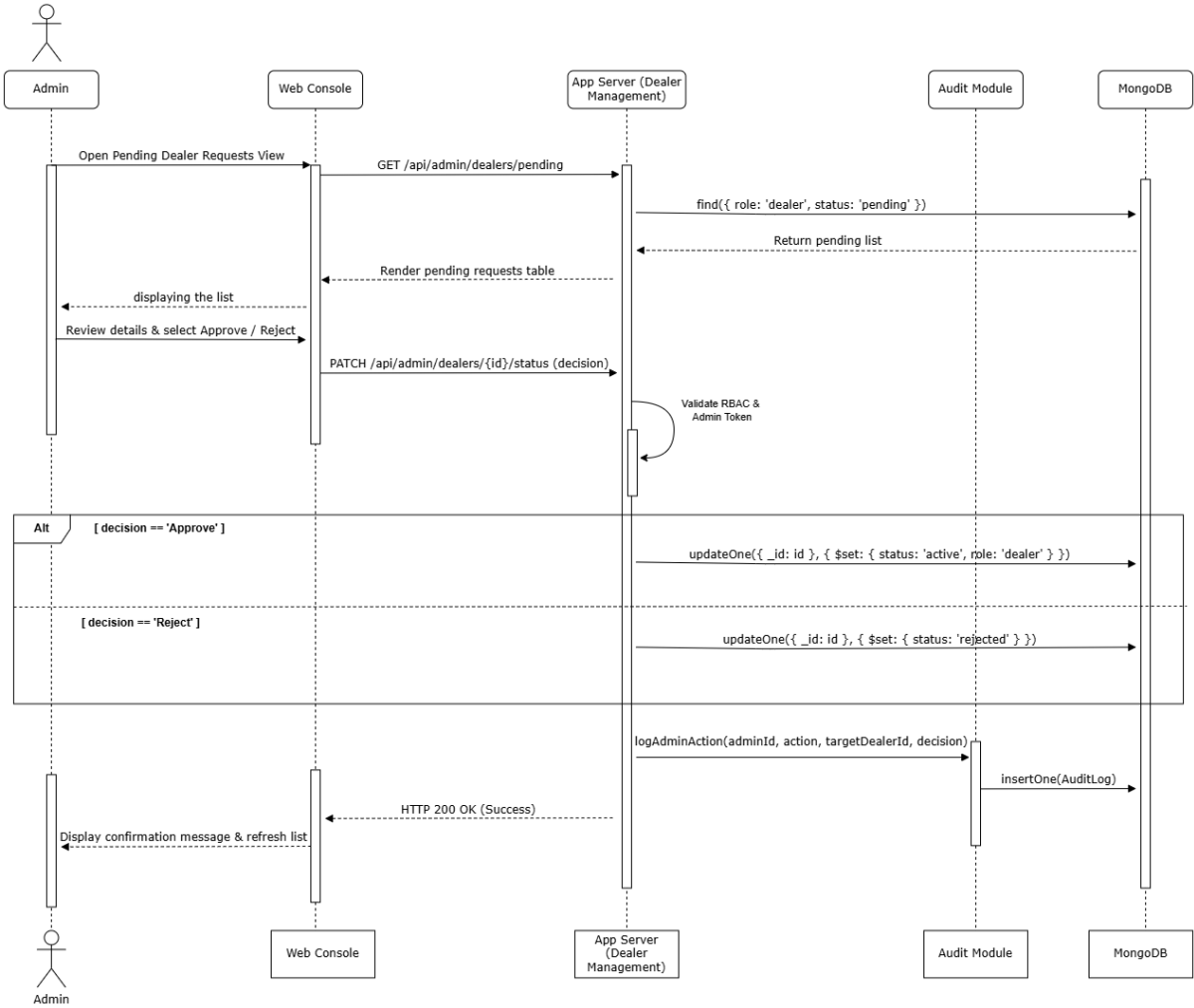


Figure 14: Administrator Approves Dealer Registration Sequence Diagram

Figure 14 shows the interaction among the Administrator, Web Console, App Server, Audit Module, and MongoDB when resolving a pending dealer-role request, with a mandatory audit entry recorded regardless of the outcome.

6.10 Notification Delivery Sequence

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

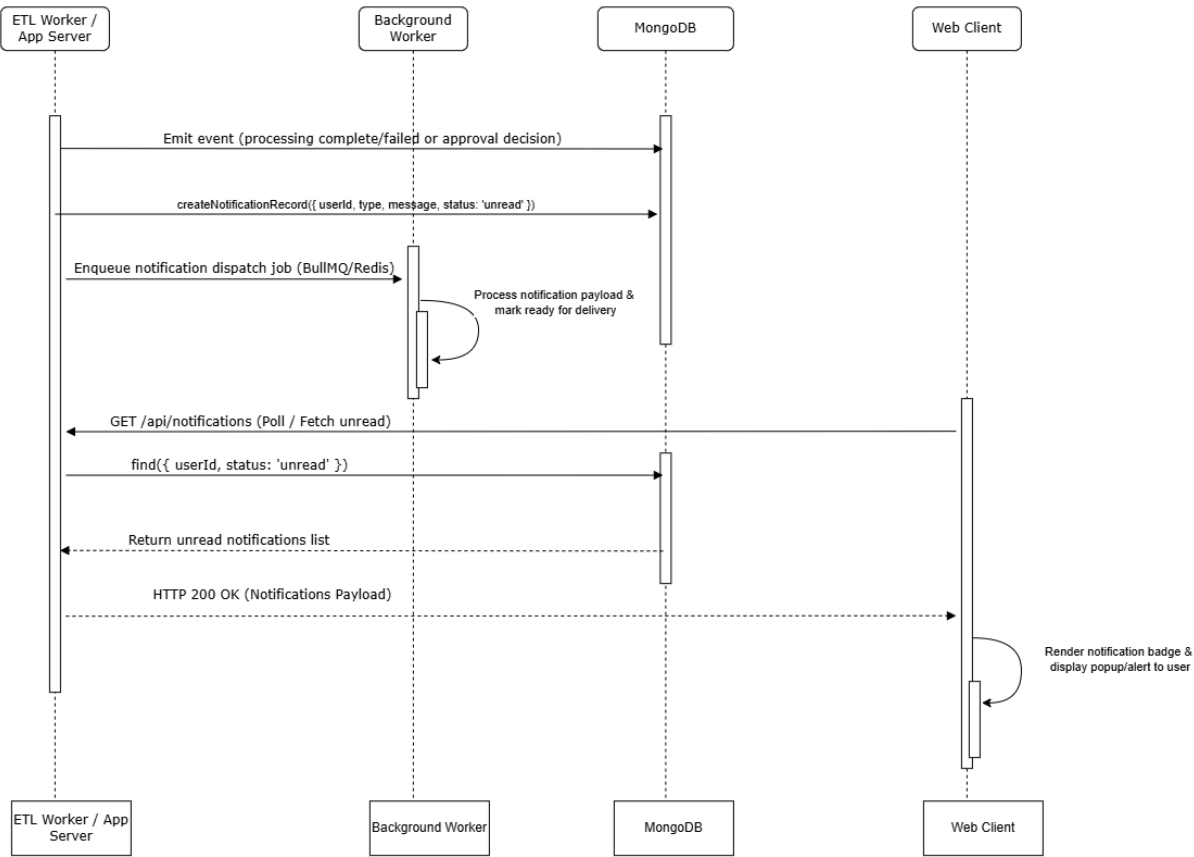


Figure 15. Notification Delivery Sequence Diagram

Figure 15 shows how a processing-outcome event (upload completion, upload failure, or dealer-approval decision) results in a Notification record that is retrieved by the affected user's web client.

6.11 Upload Job State

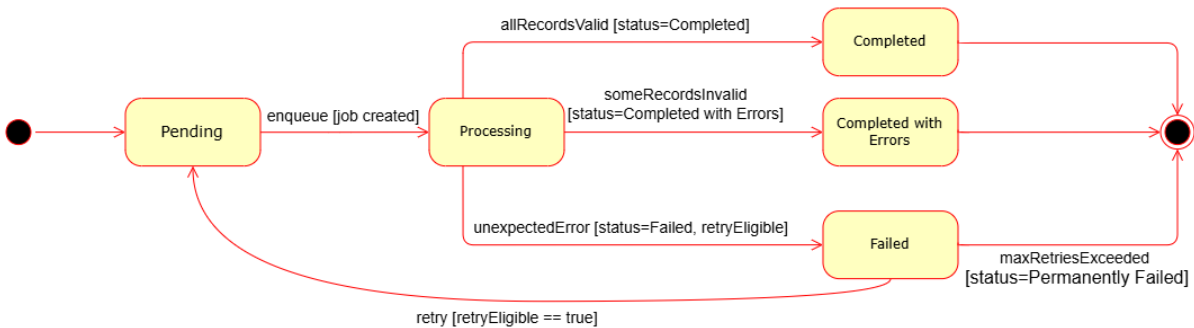


Figure 16. Upload Job State Diagram

Figure 16 shows the lifecycle of an Upload Job. A job begins in Pending once created, moves to Processing when dequeued by the ETL Worker, and ends in Completed, Completed with Errors, or

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Failed depending on the outcome of record-level validation. A Failed job may be retried, returning it to Pending.

6.12 Listing / Rejected Record State

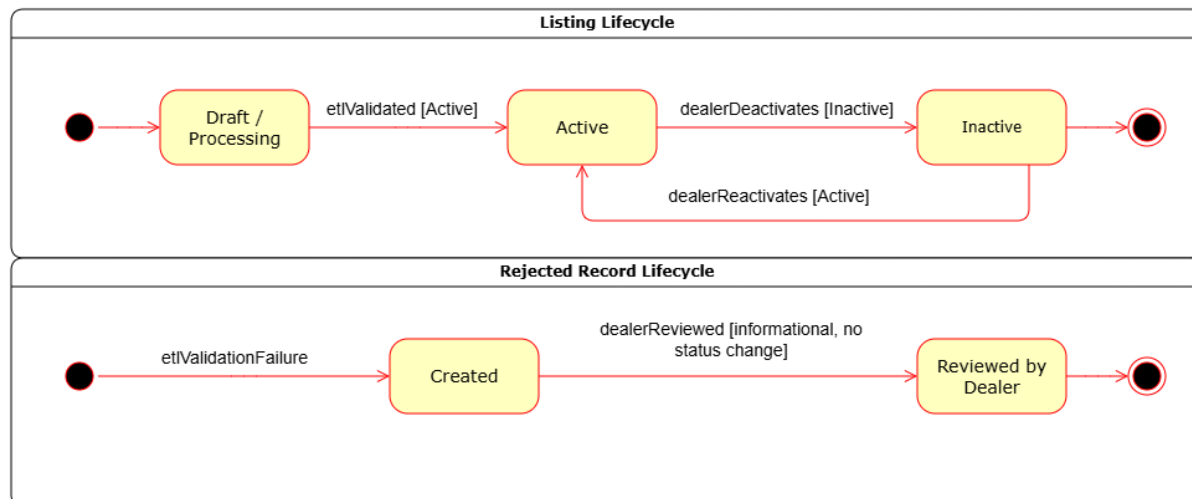


Figure 17. Listing and Rejected Record State Diagram

Figure 17 presents the lifecycle of an individual Listing once it has been created by the ETL pipeline (or directly by a dealer), together with the simpler lifecycle of a Rejected Record, which exists primarily to inform the dealer and does not itself transition into a Listing without a corrected re-upload.

7. Deployment View

The Deployment View describes the physical and cloud environments in which the system components execute and the communication paths between them. The platform is distributed across buyer/dealer/administrator client devices, a cloud-hosted (or locally containerized) web application, backend compute, a managed database, a background job queue, object storage, and external providers.

The core backend is deployed as a Node.js/Express modular monolith, while the ETL Worker runs as a separate container because inventory processing has different, burstier resource requirements than ordinary API traffic. The deployment diagram also maps the runtime processes described in the Process View to their corresponding execution nodes.

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

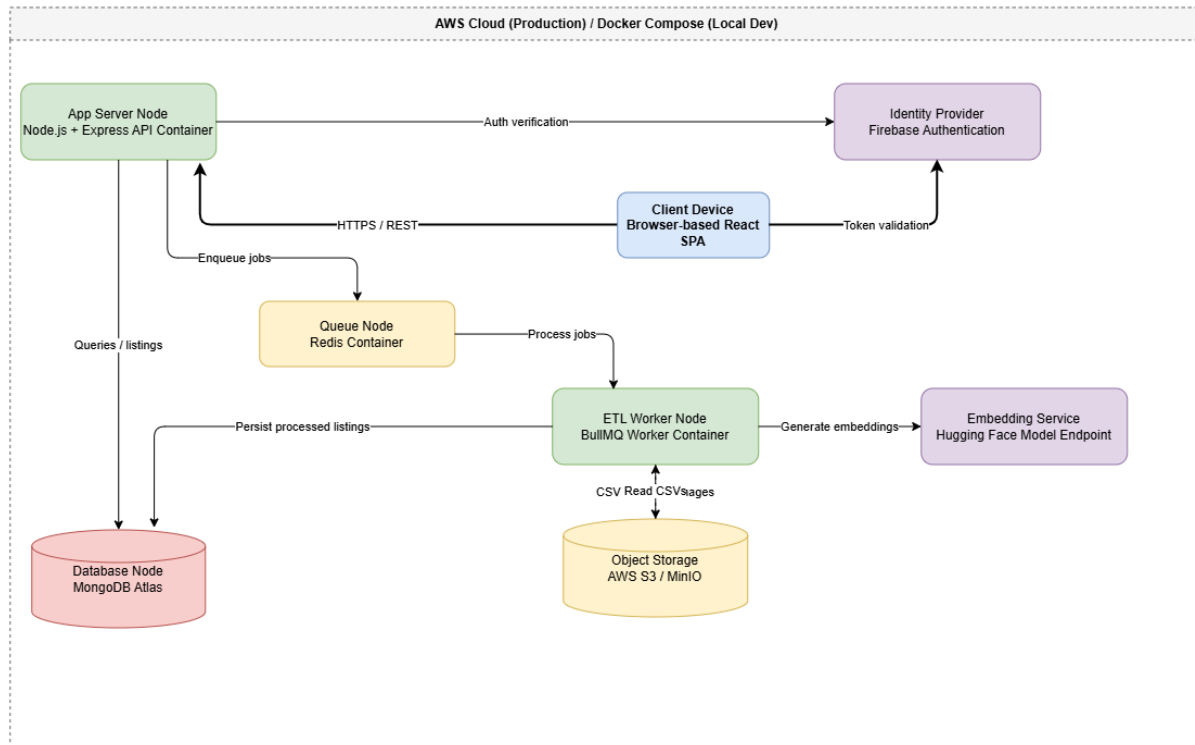


Figure 18:. Deployment Diagram — Container Nodes and Communication Paths

Node Descriptions

- Client Device — any modern desktop, tablet or mobile browser running the React/TypeScript SPA; communicates with the App Server over HTTPS/REST and with Firebase directly for authentication.
- App Server Node — a Docker container running the Node.js/Express modular-monolith API; the only node that serves client HTTP traffic.
- ETL Worker Node — a separate Docker container running the BullMQ worker process; scaled independently from the App Server as inventory volume grows.
- Queue Node — Redis, running as a Docker container locally or a managed Redis service in AWS; mediates all Upload Job messages between the App Server and the ETL Worker.
- Object Storage — MinIO (Docker container) in local development, AWS S3 in production; stores original CSV uploads and vehicle images (DC-04).
- Database Node — MongoDB Atlas (a managed cluster used identically in both environments), holding all persistent collections described in Section 9.
- Embedding Service — an external Hugging Face model endpoint used by both the ETL Worker (to embed listings) and the Search module (to embed queries).
- Identity Provider — Firebase Authentication, an external managed service; issues JWT ID tokens that the App Server verifies via the Firebase Admin SDK on every protected request.

Deployment is automated via a GitHub Actions CI/CD pipeline, which builds the App Server and ETL Worker Docker images and deploys them to the target environment (local Docker Compose for development, AWS for production), fulfilling the project's continuous-deployment objective.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

8. Implementation View

The Implementation View describes how MotorX is organized in the source code and how the major software components are grouped into layers, modules, and deployable applications. It shows how the logical architecture defined in Section 5 is mapped into implementation components.

MotorX follows a layered implementation structure consisting of the Presentation Layer, API/Controller Layer, Business/Service Layer, ETL/Processing Layer, and Data Access Layer. The backend uses a modular monolithic organization, while the ETL Worker is implemented as a separate deployable application for long-running inventory-processing tasks.

The implementation structure is designed so that higher-level components depend on lower-level components through clearly defined interfaces. The web application communicates with the backend through REST APIs, while the backend and ETL Worker access persistence, background-job, object-storage, and external-model services through the Data Access Layer.

8.1 Overview

MotorX is organized into five main implementation layers:

1. **Presentation Layer** – React + TypeScript web application.
2. **API / Controller Layer** – Express routes, controllers, validation, authentication, and authorization middleware.
3. **Business / Service Layer** – application business logic and domain rules.
4. **ETL / Processing Layer** – asynchronous inventory processing performed by the ETL Worker.
5. **Data Access Layer** – access to MongoDB, Redis/BullMQ, object storage, and external services.

The primary backend is implemented as a modular monolith. Each functional area, such as Dealer Management, Marketplace, Inventory, Search, Notifications, and Administration, has its own internal module while running within the same backend application.

The ETL Worker is deployed separately because inventory processing can take significantly longer than normal API operations. Both the backend and ETL Worker use shared infrastructure components where appropriate.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Overall Layered Component Diagram

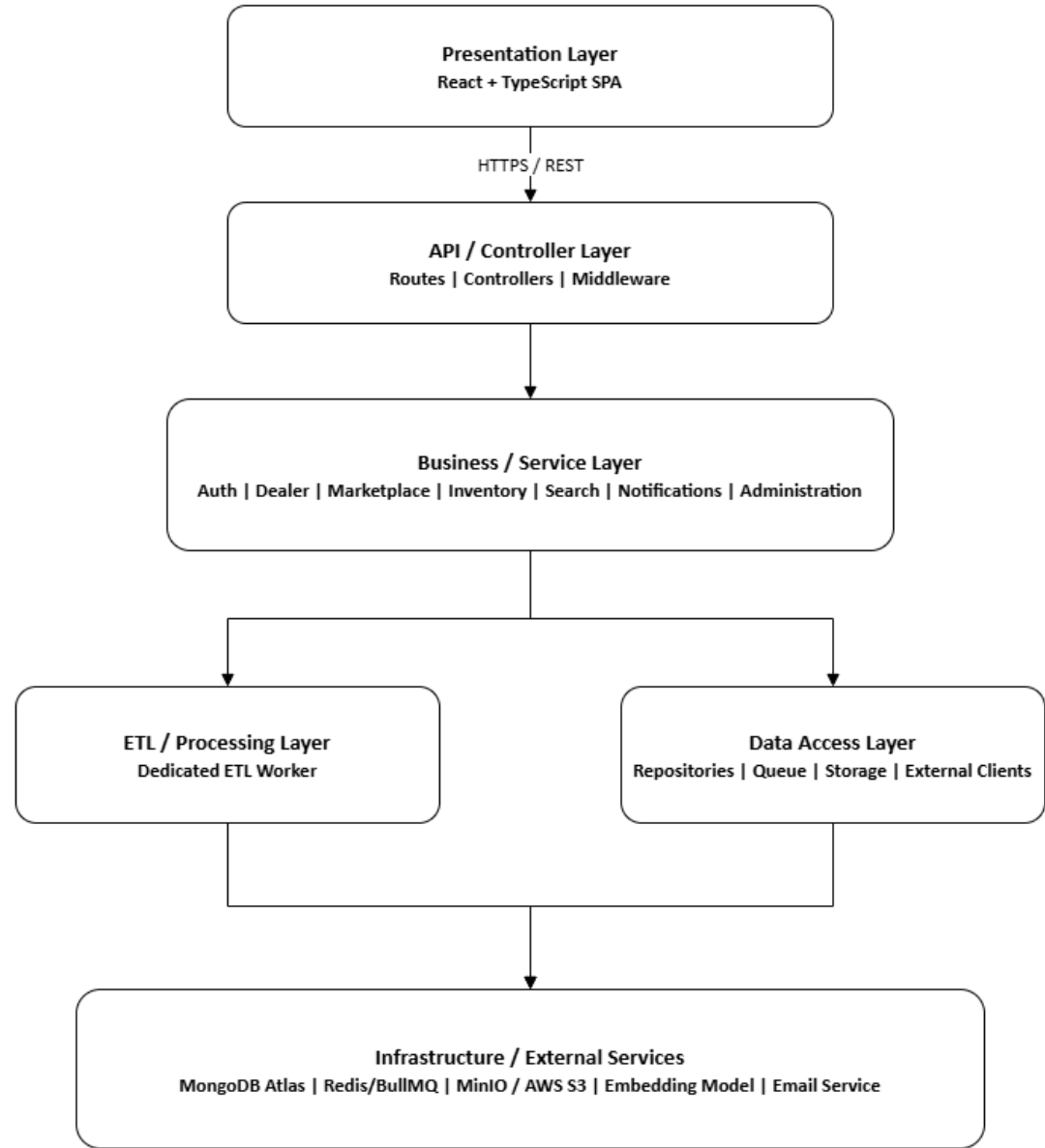


Figure 19. Overall Layered Component Diagram

Figure 19: Implementation View — Layered Component Diagram

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

8.2 Layers

8.2.1 Presentation Layer

The Presentation Layer contains the React + TypeScript single-page web application used by buyers, dealers, and administrators.

The main components of this layer are:

- Buyer Portal
- Dealer Portal
- Administrator Portal
- Authentication UI
- Shared UI Components
- API Client
- Firebase Authentication Client

The Buyer Portal contains marketplace browsing, structured search, intelligent search, and vehicle-detail interfaces.

The Dealer Portal contains dealer profile management, vehicle listing management, image management, inventory upload, upload-status monitoring, rejected-record review, and notification interfaces.

The Administrator Portal contains user and dealer management, dealer approvals, listing monitoring, system-health information, statistics, and audit information.

The Presentation Layer does not access MongoDB, Redis, or object storage directly. Application data is obtained through the backend REST API. Firebase Authentication is accessed through the Firebase client SDK for authentication and ID-token acquisition.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

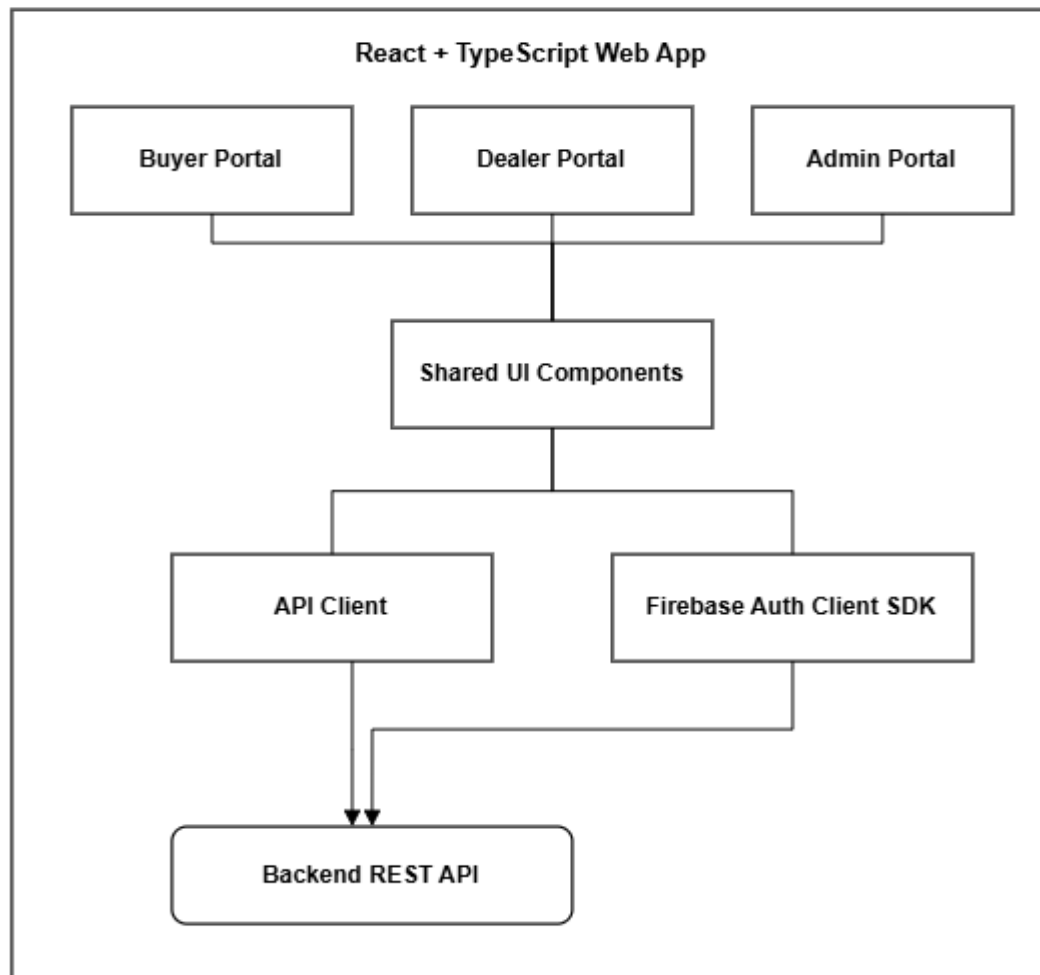


Figure 20. Presentation Layer Component Diagram

8.2.2 API / Controller Layer

The API / Controller Layer provides the HTTP interface between the web application and backend services.

It contains:

- Express Routes
- Controllers
- Request Validation
- Firebase ID-Token Verification Middleware
- RBAC Middleware
- Error-Handling Middleware

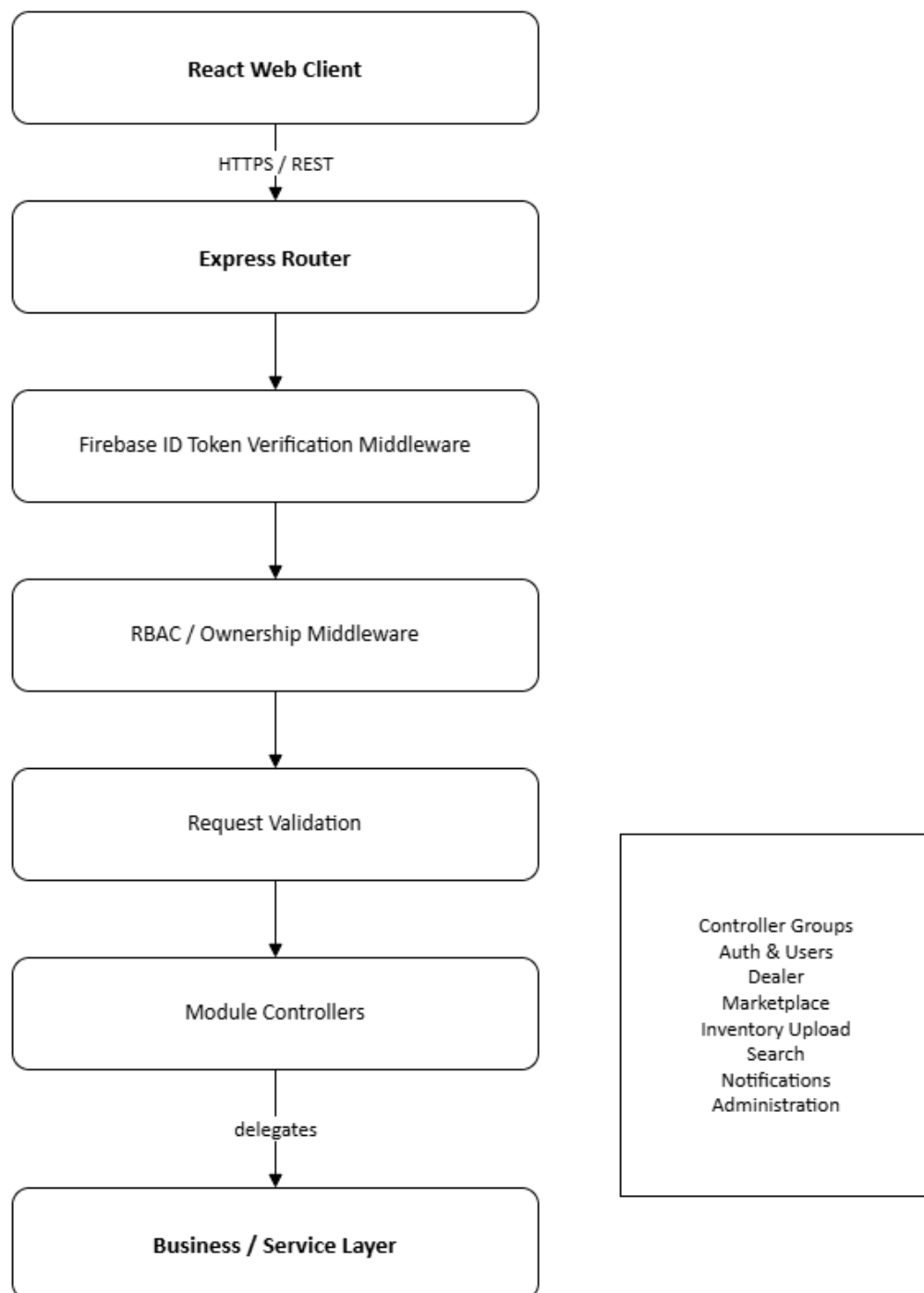
Routes are grouped according to the major backend modules, including authentication and users, dealers, marketplace, inventory upload, search, notifications, and administration.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Controllers receive HTTP requests, validate request data, and delegate processing to the appropriate service. Business rules are not implemented directly inside controllers.

Authentication middleware verifies Firebase ID tokens before protected requests are processed. Authorization middleware checks the user's MotorX application role and resource ownership where required.

API / Controller Layer Component Diagram



<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Figure 21. API / Controller Layer Component Diagram

8.2.3 Business / Service Layer

The Business / Service Layer contains the main application rules and coordinates operations between the API layer and infrastructure components.

The main service components are:

- Auth & Users Service
- Dealer Service
- Marketplace Service
- Inventory Service
- Search Service
- Notification Service
- Administration Service

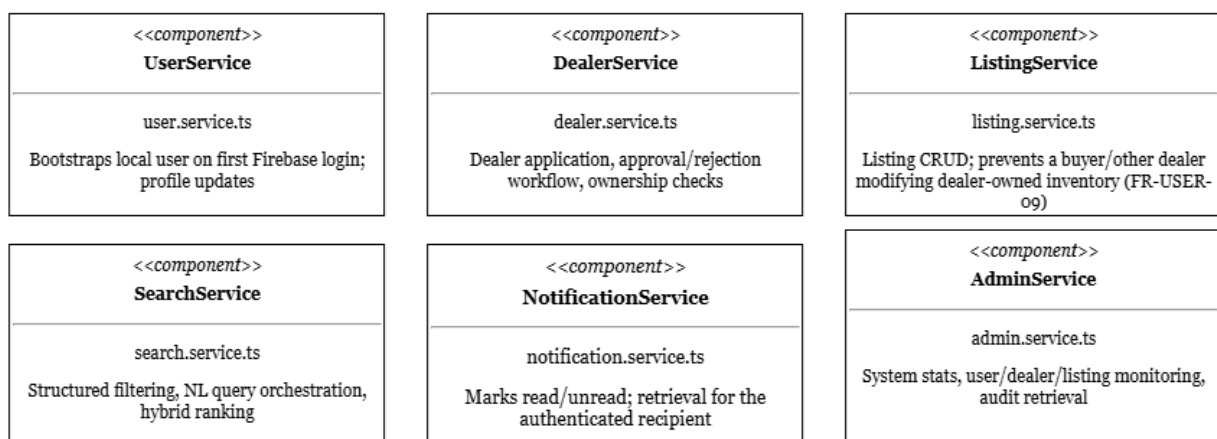
Each service is responsible for the business rules of its module.

For example, the Dealer Service manages dealer profiles and approval-related operations, while the Marketplace Service manages vehicle listings and verifies dealer ownership before changes are made.

The Search Service coordinates structured filtering, natural-language query interpretation, fuzzy matching, semantic similarity, and hybrid ranking.

The Notification Service creates in-application notifications and coordinates email-notification delivery.

Figure 23: Business / Service Layer Component Diagram



<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

8.2.4 ETL / Processing Layer

The ETL / Processing Layer executes inside the dedicated ETL Worker.

It is responsible for converting uploaded dealer inventory into valid and searchable vehicle listings.

The main processing components are:

- CSV Extractor
- Normalizer
- Validator
- Duplicate Detector
- Categorizer
- Enricher
- Embedding Generator
- Listing Writer
- Rejected Record Handler
- Upload Job Status Handler

The ETL Worker retrieves jobs from Redis/BullMQ and obtains the uploaded CSV file from object storage. Records are then processed through the ETL stages.

Valid records are stored as vehicle listings, while invalid records are stored as rejected records. Semantic embeddings are generated for searchable listing information before processing is completed.

This diagram can be very useful because your ETL pipeline is one of MotorX's main architectural features.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

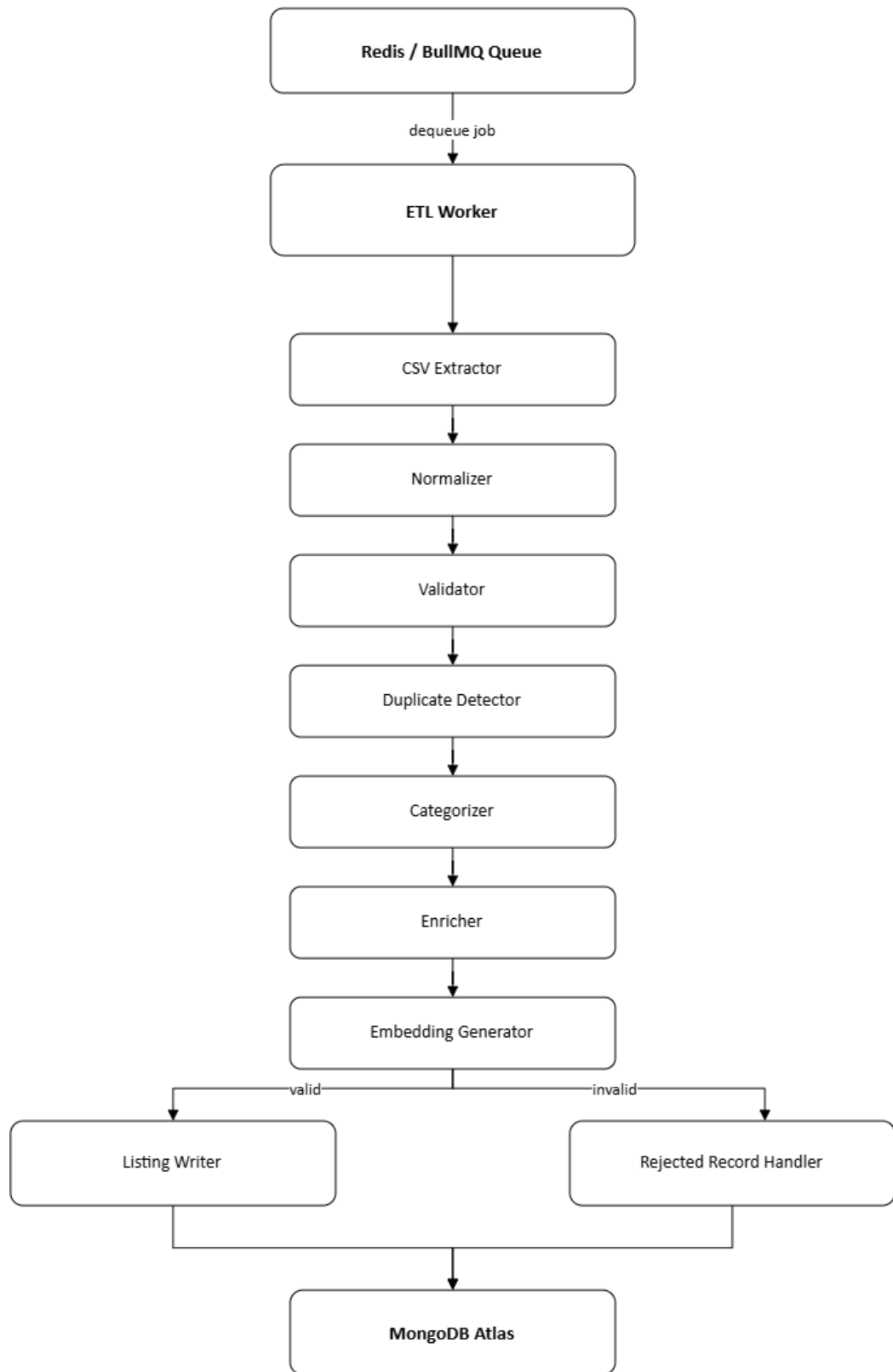


Figure 23. ETL / Processing Layer Component Diagram

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

This is different from your **ETL Activity Diagram** in Section 6. The activity diagram describes runtime behaviour; this diagram describes the **software components implementing the ETL layer**.

8.2.5 Data Access Layer

The Data Access Layer contains the components responsible for communication with persistent storage, background infrastructure, object storage, and external services.

The main components are:

- MongoDB Repositories
- MongoDB Connection Manager
- Redis/BullMQ Client
- Object Storage Client
- Embedding Model Client
- Email Service Client

MongoDB repositories provide data-access operations for users, dealers, listings, upload jobs, rejected records, notifications, and audit logs.

The Redis/BullMQ client provides background-job queue operations.

The object-storage client provides a common interface for MinIO during development and AWS S3 in production.

The embedding-model client communicates with the configured semantic-embedding model.

The email client supports delivery of important application notifications.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

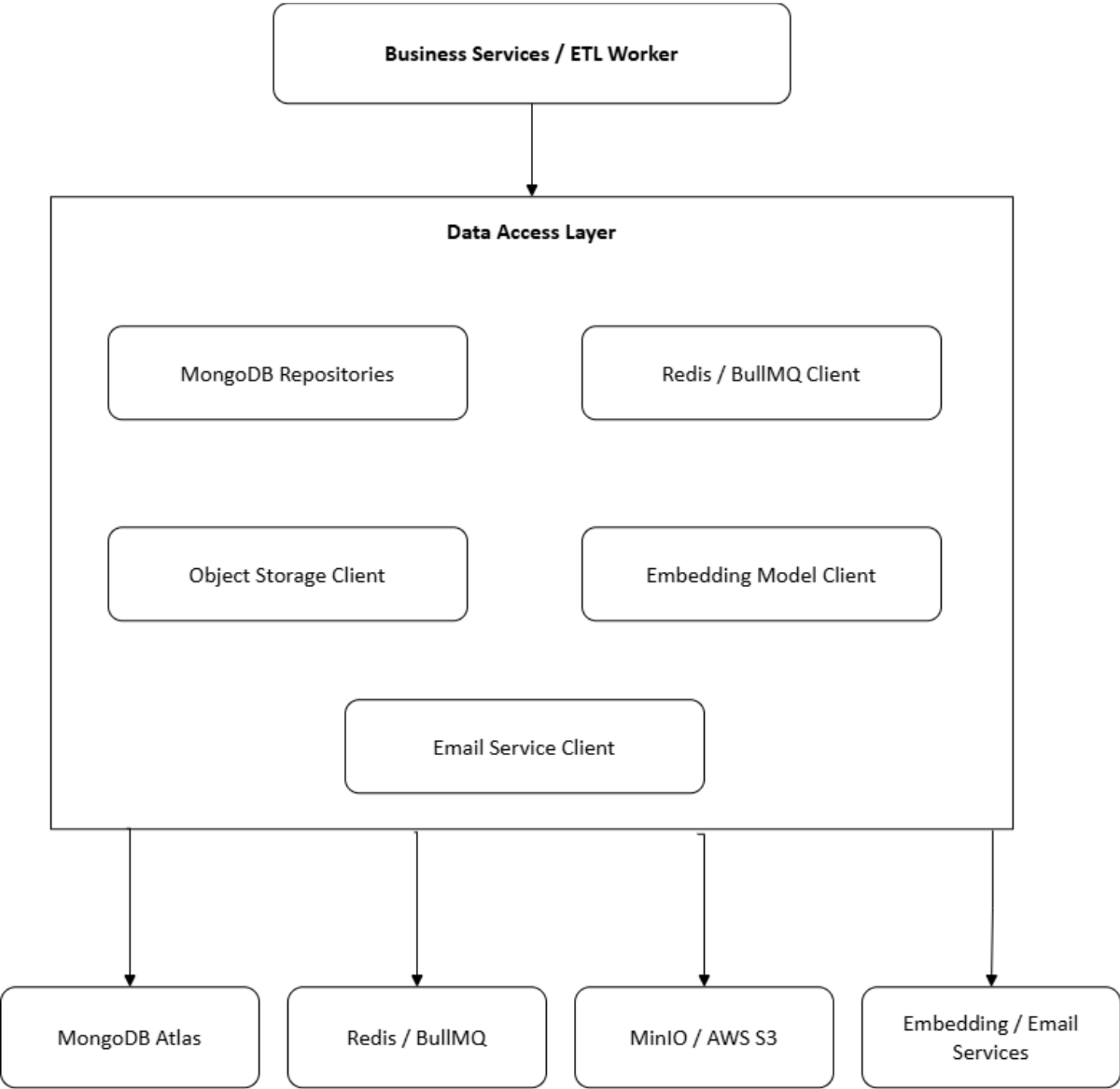


Figure 24. Data Access Layer Component Diagram

Figure 24. Data Access Layer Component Diagram

8.3 Package Structure

The implementation packages map the layered architecture to the physical source-code organization.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

The **Backend Application** is implemented as a Node.js/Express modular monolith. Its main packages correspond to the business modules identified in the Logical View:

- Auth & Users
- Dealer Management
- Marketplace
- Inventory
- Search
- Notifications
- Administration
- Shared Infrastructure

Each business module may contain internal routes, controllers, services, schemas, and repository-related components according to its responsibilities.

The **ETL Worker** is a separate deployable application containing the inventory-processing pipeline and supporting components. Its internal packages include:

- job handling
- pipeline stages
- embedding
- validation
- integrations
- configuration

Shared implementation packages provide database access, background-job infrastructure, common contracts, configuration, and external-service integrations.

The **Web Application** is implemented separately using React and TypeScript and communicates with the backend through REST APIs over HTTPS.

Dependencies follow the implementation-layer rules. Higher layers may depend on lower layers, while lower layers do not depend on presentation or API components.

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

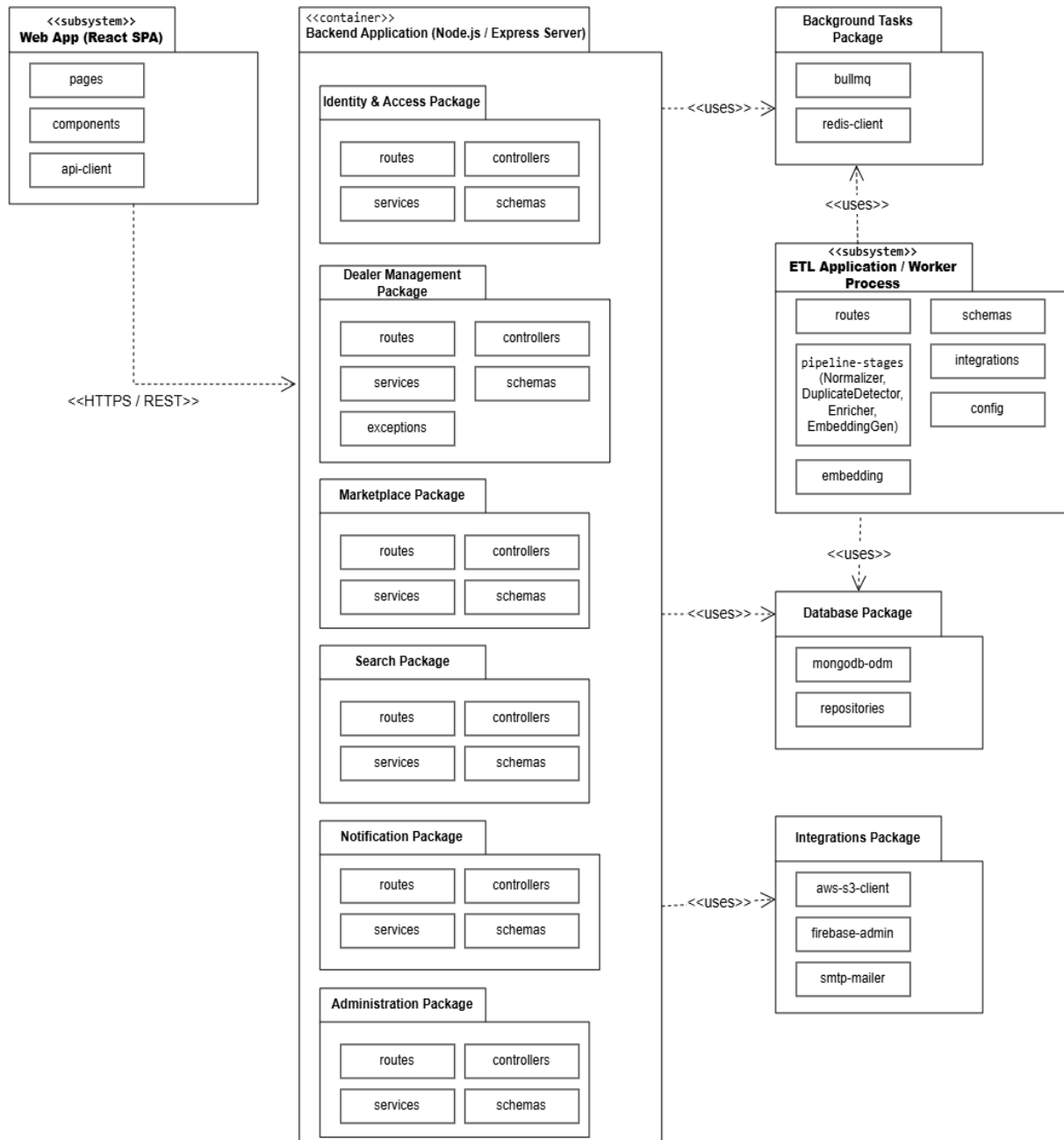


Figure 20: Backend Package Diagram

9. Data View (Optional)

9.1 Entity-Relationship Model

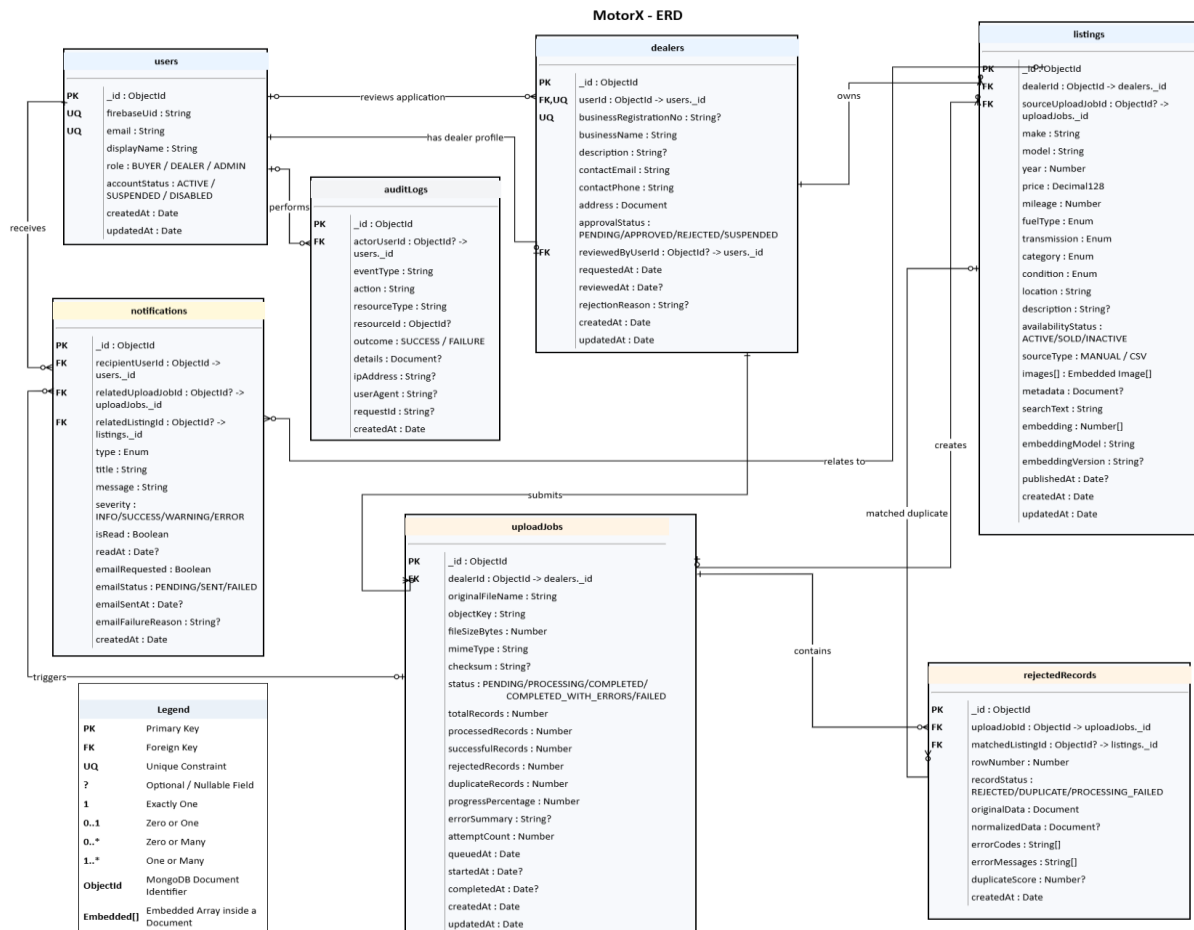


Figure 21: Entity-Relationship Diagram

Persistent Data Storage

MotorX uses **MongoDB Atlas** as its primary persistent database, with data logically separated into the following collections. Original inventory files and vehicle images are stored in **object storage** (MinIO during development and AWS S3 in production), while MongoDB stores the corresponding metadata and file references.

Collection	Main Data / Purpose
Users	User ID, Firebase UID, role, account status, profile information, timestamps.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Dealers	Dealer ID, associated user ID, business information, contact details, approval status, timestamps.
Listings	Listing ID, dealer ID, vehicle attributes, description, condition, location, embedded image metadata, availability, metadata, semantic vector embedding, timestamps.
Upload Jobs	Upload ID, dealer ID, uploaded file reference, processing status (Pending, Processing, Completed, Completed with Errors, Failed), timestamps, processing statistics.
Rejected Records	Rejected record ID, upload job ID, original row data, validation errors, rejection reason.
Notifications	Notification ID, recipient user ID, notification type, message, read status, email delivery status, timestamp.
Audit Logs	Audit log ID, user ID (where applicable), operation type, affected resource, timestamp, activity details.

Relationships: A Dealer references exactly one User. A Listing references exactly one Dealer. An Upload Job references exactly one Dealer and may produce zero or more Listings and zero or more Rejected Records. A Notification references exactly one recipient User.

Indexes shall be maintained on frequently queried fields including make, model, price, year, fuel type, transmission, category, location, dealer ID, and availability to support efficient structured filtering. Additional indexes shall support dealer-specific inventory retrieval, upload job tracking, notification retrieval, and user authentication using dealer ID, upload job ID, recipient user ID, and Firebase UID.

9.2 Data-Store Responsibilities

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing	Version: 1.0
Software Architecture Document	Date: 08/08/2026
SAD-v1.0	

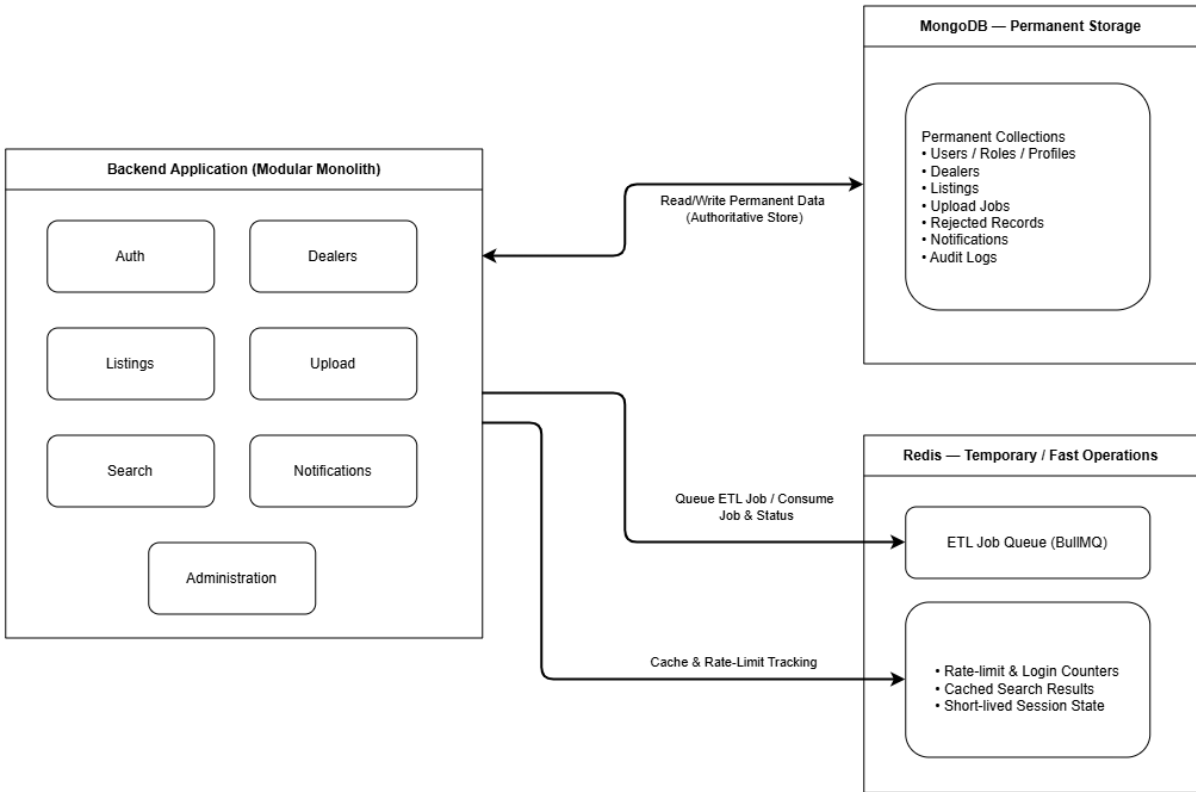


Figure 22: MongoDB and Redis Responsibility Diagram

The original uploaded CSV files and vehicle images are intentionally stored outside MongoDB, in Object Storage (S3/MinIO), with only a reference/URL kept in the corresponding document — this keeps the primary database compact and keeps large binary content on infrastructure designed for it. Redis is limited to temporary and recoverable data (queue state, counters, caches); the loss of Redis data must never result in the loss of a finalized Listing or Upload Job record, since MongoDB remains the sole authoritative store for permanent data.

10. Size and Performance

The size and performance requirements of MotorX influence several important architectural decisions. The system must support normal marketplace operations, intelligent vehicle search, dealer inventory management, and bulk inventory processing without allowing one type of workload to significantly affect another.

The main performance-related architectural considerations are API response time, search response time, bulk inventory processing, future system growth, and system availability.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

10.1 API Responsiveness

Normal API operations shall return successful responses within **2 seconds for at least 95% of requests under expected load** (PSR-01). These operations include common activities such as retrieving vehicle listings, viewing vehicle details, accessing dealer information, managing profiles, and checking inventory upload status.

To support this requirement, the main Backend Application handles interactive API requests separately from long-running ETL processing. Controllers are kept lightweight and delegate application logic to the appropriate services. Database queries shall also use appropriate indexes for commonly accessed fields.

The separation between normal API processing and background inventory processing is important because a large CSV upload should not reduce the responsiveness of marketplace operations for other users.

10.2 Search Performance

MotorX supports both structured vehicle search and intelligent natural-language search. These search methods have different processing requirements and therefore have different performance targets.

Standard structured searches shall normally return results within **2 seconds under expected operating conditions** (PSR-02). Structured searches primarily use indexed vehicle attributes such as make, model, price, year, mileage, fuel type, transmission, category, location, and availability.

Intelligent semantic searches shall normally return results within **5 seconds**, excluding temporary delays caused by the external embedding model or related model infrastructure (PSR-03).

The intelligent search process may perform several operations, including:

- natural-language query analysis;
- extraction of structured constraints;
- fuzzy matching;
- query embedding generation;
- semantic similarity search;
- hybrid relevance ranking;
- sorting; and
- pagination.

The architecture therefore keeps search functionality inside a dedicated Search module rather than distributing search logic across controllers or other business modules. Structured filtering is applied where appropriate before semantic ranking so that unnecessary candidate listings can be reduced before more expensive relevance calculations are performed.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

This design allows normal filter-based searches to remain simple while still supporting the additional processing required by intelligent search.

10.3 Bulk Inventory Processing

Bulk dealer inventory processing is one of the most resource-intensive operations in MotorX.

The system shall be designed to process approximately **5,000 vehicle records within 2 minutes in a defined test environment**, subject to enrichment and embedding complexity (PSR-05).

Processing an inventory file may involve several operations for every vehicle record, including:

1. CSV extraction;
2. data normalization;
3. validation;
4. duplicate detection;
5. categorization;
6. data enrichment;
7. semantic embedding generation; and
8. storage of valid or rejected records.

Performing all of these operations during the original HTTP upload request could result in long response times or request timeouts. For this reason, MotorX uses asynchronous inventory processing.

After an accepted CSV file is stored in object storage and an Upload Job is created, a processing job is placed in the Redis/BullMQ queue. The dedicated ETL Worker retrieves the job and performs the processing independently from the main Backend Application.

The ETL Worker processes inventory records in manageable batches (FR-ETL-06). Batch processing reduces the amount of data that must be held in memory at one time and provides better control over large inventory files.

This architecture ensures that a dealer uploading a large inventory file does not block normal API requests from buyers, other dealers, or administrators.

10.4 Background Job Performance

Redis and BullMQ are used to separate background inventory-processing work from interactive application requests.

The Backend Application is responsible for accepting the upload and placing the processing job into the queue. The ETL Worker is responsible for retrieving and processing that job.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Upload Jobs use processing states such as:

- Pending;
- Processing;
- Completed;
- Completed with Errors; and
- Failed.

These states allow processing to continue asynchronously while the dealer can use other parts of the application.

Recoverable background-processing failures may also be retried according to the ETL processing rules. Failed jobs are recorded so that processing problems can be diagnosed without affecting unrelated uploads.

10.5 Database Size and Retrieval

MongoDB Atlas is used as the primary persistent database. The amount of stored data is expected to increase as more dealers, vehicle listings, inventory uploads, notifications, rejected records, and audit records are created.

Vehicle listings are particularly important from a performance perspective because they are frequently accessed by both marketplace and search operations.

Appropriate database indexes shall therefore be created for frequently queried attributes such as:

- make;
- model;
- price;
- year;
- mileage;
- fuel type;
- transmission;
- category;
- location;
- dealer identifier; and
- availability status.

Additional indexes shall support frequently accessed relationships and identifiers such as Firebase UID and Upload Job ID.

The purpose of these indexes is to prevent common marketplace and dealer operations from requiring unnecessary full-collection scans as the amount of stored data increases.

Semantic vector information associated with vehicle listings shall also be stored in a form that supports the selected semantic similarity search mechanism.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

10.6 Object Storage

Uploaded CSV files and vehicle images are stored separately from the main MongoDB database.

MotorX uses **MinIO during local development** and **AWS S3 in production** for object storage.

Separating binary and uploaded file content from application metadata prevents large files from unnecessarily increasing the size of database documents. MongoDB stores the metadata and references required by the application, while the actual files remain in object storage.

This also allows the object-storage implementation to be changed according to the deployment environment without requiring changes to the main business logic.

10.7 Scalability

The architecture shall support future growth in the number of users, registered dealers, vehicle listings, searches, and inventory-processing operations without requiring major architectural changes (PSR-07).

The modular monolithic backend provides clear internal module boundaries while keeping deployment manageable for the current project. At the same time, the ETL Worker is already separated from the Backend Application because its workload has different performance characteristics.

This means that interactive application processing and background inventory processing can be managed independently where required.

Redis/BullMQ also provides a queue between the Backend Application and ETL Worker. Therefore, temporary increases in inventory uploads do not require every uploaded file to be processed inside the corresponding HTTP request.

The current architecture focuses on realistic project requirements rather than introducing unnecessary distributed infrastructure.

10.8 Availability

The production system shall target at least **99% monthly availability**, excluding scheduled maintenance (RR-01).

MotorX depends on several external and managed services, including Firebase Authentication, MongoDB Atlas, object storage, and the selected embedding infrastructure. Availability of these services can therefore affect individual system capabilities.

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

Failures should be handled according to the affected operation. For example, temporary ETL processing failures should not make the buyer marketplace unavailable, and a temporary embedding-model failure should not cause unrelated dealer or administrative operations to fail.

Application and worker errors shall be logged so that operational problems can be identified and diagnosed.

11. Quality

Beyond raw functionality, the architecture was shaped to support the following quality attributes:

Reliability

Partial-success semantics in the ETL pipeline (FR-ETL-13 to FR-ETL-18) ensure that a single malformed row never discards an entire valid upload. Upload Jobs, Listings and Rejected Records are explicitly linked (RR-04) for full traceability, and failed background jobs are recorded and retryable (RR-06/RR-07).

Scalability

Separating the ETL Worker from the API process, combined with the modular monolith's clean internal boundaries, allows either side to be scaled independently, and provides a low-risk migration path toward extracting individual modules into services later if the 100,000-listing / 10,000-user targets are exceeded.

Security

Firebase Authentication issues short-lived signed JWTs; the backend never stores passwords (PSR-11), and every protected route is guarded by token verification plus RBAC (FR-USER-04 to FR-USER-11). All production traffic uses HTTPS (PSR-10), and secrets are supplied through environment/secret-management mechanisms rather than being hard-coded (SR-10).

Maintainability / Supportability

The layered Implementation View (Section 8) separates presentation, API, business logic, ETL and data-access concerns (SR-02/SR-03), and the Logical View's package boundaries mirror the SRS requirement subsets one-to-one, so a change to one functional area (e.g., Search) has minimal ripple effect on others.

Portability

Docker and Docker Compose containerize every custom component (App Server, ETL Worker, and local MinIO/Redis for development), so the identical container images run unmodified on a developer's laptop and on AWS (SR-08/SR-09), fulfilling the project's cloud-native portability objective.

Usability

The hybrid hierarchy of search — plain filters for buyers who know what they want, natural language for buyers who do not — is reflected architecturally in the Query Analyzer sitting in front of, rather

<i>Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing</i>	<i>Version: 1.0</i>
<i>Software Architecture Document</i>	<i>Date: 08/08/2026</i>
<i>SAD-v1.0</i>	

than replacing, the structured Search Module, so both interaction styles share the same ranking and pagination infrastructure (UR-05 to UR-08).

12. References

- [1] Group 23, "Project Proposal — Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing," PID 11, University of Moratuwa, 05/Jul/2026.
- [2] Group 23, "MotorX — Software Requirements Specification," Version 1.0, University of Moratuwa, 02/Aug/2026.
- [3] React, "React – A JavaScript library for building user interfaces." [Online]. Available: <https://react.dev> (Accessed 05/Aug/2026).
- [4] Node.js Foundation, "Node.js — JavaScript Runtime." [Online]. Available: <https://nodejs.org> (Accessed 05/Aug/2026).
- [5] MongoDB Inc., "MongoDB Documentation." [Online]. Available: <https://www.mongodb.com/docs> (Accessed 05/Aug/2026).
- [6] Google, "Firebase Authentication Documentation." [Online]. Available: <https://firebase.google.com/docs/auth> (Accessed 05/Aug/2026).
- [7] Docker Inc., "Docker Documentation." [Online]. Available: <https://docs.docker.com> (Accessed 05/Aug/2026).
- [8] Amazon Web Services, "AWS Documentation." [Online]. Available: <https://docs.aws.amazon.com> (Accessed 05/Aug/2026).
- [9] Diagrams in this document (use-case, package/class, sequence, activity, deployment, layered component) were drawn programmatically using Python/Matplotlib, following standard UML notation conventions.